SOFTWARE DESIGN DESCRIPTION

Enterprise Cloud-Ready Business Management System (BMS)

Prepared in Accordance with IEEE Standard 1016-2009
Systems and Software Engineering — Design Descriptions

System: Business Management System (BMS)
Author: Maxwell Gachuhi
Role: Junior Software Developer
Version: 1.0
Date: July 2026
Status: Initial Architecture Release

REVISION HISTORY

The revision history log records all modifications, architectural updates, and peer-review enhancements performed on this document during its engineering lifecycle.

Table 1.2: Revision History

Version	Date	Author	Description of Changes
1.0	July 8, 2026	Maxwell Gachuhi	Initial structural design, architectural framework definition, and repository schema specification baseline.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

This Software Design Description (SDD) establishes a comprehensive, immutable technical specification and structural design blueprint for the enterprise cloud-ready Business Management System (BMS). Formulated in structural conformance with the IEEE 1016-2009 standard, this document operationalizes the high-level functional and non-functional requirements compiled in the System Requirements Specification (SRS) into highly granular, physical, component-level architectural specifications. The structural artifacts embedded within this text provide concrete execution patterns, structural component dependencies, database entity relationships, interface paradigms, and deployment configurations required for production-grade development.

The system is specifically engineered to abstract, centralize, and automate multi-tenant business profiles, role-based workforce compliance, customer and supplier value chains, real-time programmatic inventory accounting, synchronized purchase orders, and multi-channel Point of Sale (POS) transactional computations. By providing an explicit mapping of structural and behavioral mechanics, this document guarantees architectural predictability, security compliance, linear system scalability under heavy transactional stress, and strict implementation alignment across backend and frontend engineering modules.

1.2 Scope

The architectural scope of this SDD encompasses all software layout designs, infrastructural topology configurations, and cross-cutting security layers required to compile, provision, and maintain the Business Management System (BMS). The platform operates as a secure, distributed, multi-tenant web application utilizing a bifurcated client-server infrastructure. The runtime space is partitioned into a decoupled frontend user interface tier, an isolated asynchronous backend application service engine, a localized memory-caching layer, and an enterprise relational database persistence core.

Specifically, the system design details the structural layout implementation for the following subsystems:

- **Business Profiles & Tenancy Management:** Logical structures for isolation of client business metadata, functional localization configurations, tax registration parameters, and baseline financial settings.
- **Identity Security & RBAC:** High-security execution contexts governing JSON Web Token (JWT) workflows, password decryption-resistant hashing pipelines, role-to-permission mapping matrices, and localized request-intercepting authorization guards.
- **Supply Chain Management (CRM & SRM):** Entity pipelines and transactional records tracking customer demographic metrics and supplier fulfillment metrics.
- **Inventory & Lifecycle Management:** Concurrent item tracking mechanics, programmatic variant categories, real-time inventory adjustments, and double-entry auditing for inventory transaction histories.

- **Point of Sale (POS) Transaction Sub-system:** Asynchronous order processing, race-condition resistant quantity computations, physical digital receipt streaming, and third-party payment infrastructure callbacks.
- **Reporting and Business Intelligence:** High-performance read-replica analytical indexing, multi-dimensional relational aggregation, and deterministic chronological data reporting.

1.3 Intended Audience

This technical description is designed to serve as a high-fidelity reference manual for a diverse group of specialized stakeholders within the engineering lifecycle. Lead System Architects and Backend Engineering Teams utilize this specification as an immutable map to build and structure NestJS controllers, services, database abstractions, and optimization layers. Frontend Engineers utilize the specified application interfaces and domain contracts to construct typed Next.js applications, matching visual states exactly with the underlying entity models. Database Administrators (DBAs) utilize Chapter 4 to verify index distributions, partition alignments, primary and foreign key constraints, and performance parameters within PostgreSQL 18. DevSecOps Specialists rely on the infrastructure specifications to construct reproducible Docker configurations, Nginx caching logic, and isolated environment distributions. Finally, Quality Assurance (QA) Automation Engineers extract state logic to formulate deterministic end-to-end integration test suites.

1.4 Definitions, Acronyms and Abbreviations

To preserve absolute technical clarity across all chapters, the following specialized terms, acronyms, and operational vocabularies are defined under standard industry definitions:

Table 1.1: Acronyms and Technical Definitions

Term / Acronym	Technical Definition and Architectural Context
BMS	Business Management System. The comprehensive distributed software suite under design within this architectural framework.
JWT	JSON Web Token. An open standard (RFC 7519) defining a compact, self-contained method for securely transmitting claims between client and server as a JSON object.
RBAC	Role-Based Access Control. An absolute security access mechanism mapping explicit granular permissions directly to roles, which are subsequently granted to authenticated users.
ORM	Object-Relational Mapping. An abstract data access layer mapping strongly typed object structures into highly optimized relational SQL statements. Represented here via Prisma ORM.
POS	Point of Sale. The computing sub-system responsible for executing checkout logic, managing customer ledger interactions, and updating inventory records.
SME	Small and Medium Enterprise. The targeted commercial organizational demographic characterized by complex, high-throughput, localized transactional activities requiring unified system automation.

1.5 References

The architectural paradigms, verification rules, and interface specifications contained in this document are mapped to and comply with the following structural frameworks and official technical documentation standards:

- IEEE Standard 1016-2009: IEEE Standard for Information Technology — Systems and Software Engineering — Software Design Descriptions.
- PostgreSQL 18 Core Documentation: High-Concurrency Transaction Isolation Level and Indexing Optimization Manuals, 2026.
- NestJS Framework Architecture: Modular Application Design Patterns and Declarative Dependency Injection Framework Guides, 2025.
- Next.js App Router Paradigms: Server Component Optimization, Streaming Architectures, and Incremental Static Regeneration Standards, 2026.

1.6 Document Organization

This design document follows a strictly logical progression designed to facilitate rapid architectural decoding. Chapter 1 establishes structural scope and terminology. Chapter 2 details high-level design constraints, system goals, and cross-cutting architectural principles. Chapter 3 provides an abstract dissection of the system component architecture and package distributions. Chapter 4 establishes the absolute relational layout structure, data types, normalization levels, and integrity rules. Chapter 5 formalizes system behavior through structured Use Case, Activity, Sequence, and State transitions. Chapter 6 analyzes the underlying code structural representations. Chapter 7 through Chapter 10 clarify interface engineering, rigorous application defense systems, containerized infrastructure patterns, and rationale matrices for technology selections. Finally, Chapter 11 maps out long-term technological expansions for the system.

2. DESIGN CONSIDERATIONS

2.1 Design Goals

The software design of the Business Management System (BMS) is governed by a set of foundational technical objectives. These design goals ensure that the resulting system meets production standards for enterprise software, maximizing lifecycle efficiency and minimizing long-term engineering overhead.

High Availability and Fault Tolerance

The system architecture must maintain an annual uptime profile of 99.95%, excluding scheduled maintenance windows. This goal requires the design to completely isolate system components so that the failure of an auxiliary module — such as reporting or digital receipt generation — does not impact core transactional capabilities like Point of Sale operations or database persistence. Fault tolerance is reinforced at the software level through structured exception filtering, asynchronous fallback queues, and multi-node service redundancy.

Transactional Integrity and Strict Data Consistency

Because the system orchestrates financial data, inventory valuation ledger records, and legal tax compliance entries, the architecture enforces absolute transactional atomicity, consistency, isolation, and durability (ACID). Every state adjustment across product metrics, inventory counts, or sales values must be evaluated in isolated, database-level transactions. The system prevents race conditions, write-skew anomalies, and phantom reads under concurrent multi-user environments by utilizing strict PostgreSQL isolation levels and row-level locking patterns.

Low-Latency Operational Performance

The Point of Sale user interface demands near-instantaneous query feedback to ensure high operational throughput at physical checkout points. The backend engineering target mandates that 95% of standard transactional operations (API read/write cycles) must execute with an end-to-end network round-trip time under 200 milliseconds. This performance threshold is achieved through database index optimization, aggressive Redis in-memory key-value caching, optimized raw database queries via the ORM layer, and Next.js server-side component streaming.

Seamless Horizontal and Vertical Scalability

The underlying computing topology must support seamless scalability to handle growing numbers of multi-tenant businesses, physical retail branches, and concurrent customer checkouts. The system avoids tightly coupled execution dependencies so that stateless application container layers can scale horizontally across distributed cloud environments via orchestrators. The persistence tier is designed for rapid vertical scale-ups and reads are offloaded using dedicated replication clusters.

Extensible Modular Customization

To ensure the system can accommodate future business needs without requiring complete core refactoring, the software design uses modular design patterns. Domain contexts — such as inventory, billing, or access control — are decoupled into isolated packages that interact through

strict, contractually defined internal interfaces. This clear separation of concerns ensures that future feature expansions, such as integrating AI sales forecasting or automated accounting, can be added with minimal impact on existing code structures.

2.2 Assumptions

The architectural patterns and technical decisions specified throughout this design document depend on several contextual baseline assumptions. Should any of these assumptions be violated during implementation or production deployment, parts of the system design may require revisions.

Persistent Network Interconnectivity

It is assumed that target small and medium enterprises deploy this platform within environments supported by high-speed, stable broadband internet connectivity. The core Point of Sale and inventory tracking workflows are designed as cloud-native applications that require uninterrupted, low-latency access to the centralized backend NestJS application cluster and the PostgreSQL database engine.

Modern Browser Runtime Execution

The client application environment is assumed to run on modern, standard internet browsers that fully support the ECMAScript 6+ standard, hardware-accelerated CSS rendering, advanced DOM manipulation models, and secure cryptographic storage mechanisms (such as SessionStorage and local Cookie policies). Supported runtimes include current enterprise versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari. Legacy web engines lacking modern asynchronous processing capabilities are excluded from target compatibility metrics.

Centralized Singly Rooted Database Persistence

The architecture assumes that all tenant data across all organizational profiles can be securely isolated and maintained within a single, highly optimized relational PostgreSQL database cluster using strict logical filtering (Tenant-ID separation columns). While the design accommodates independent read-replicas for analytics, it assumes that a single, unified database schema serves as the single source of truth for all transactional writes.

2.3 Constraints

The implementation of the Business Management System is bounded by concrete technical, operational, regulatory, and architectural constraints that limit design flexibility but guarantee system security and consistency.

Hardware and Memory Limits

The application servers are configured to execute inside isolated Linux Docker containers limited to standard budget cloud instance allocations. The backend runtime execution space is restricted to a maximum of 2 Gigabytes of RAM per container node, while the Redis cache instance operates under strict memory eviction rules (Least Recently Used policy) with an upper memory threshold of 1 Gigabyte. The database schema design must minimize row overhead and use optimized data types to avoid excessive memory consumption.

Relational Engine Version Bounds

The data persistence layer is strictly bound to PostgreSQL version 18.0 or higher. The design relies on specific modern relational engine capabilities, including advanced B-Tree index optimizations, jsonb structural manipulation primitives, and optimized multi-table common table expressions (CTEs). The architecture cannot be backward-migrated to legacy SQL environments or switched to non-relational document databases without invalidating the core transaction design.

Data Privacy and Security Regulations

The system must comply with data protection regulations regarding the processing, retention, and encryption of commercially sensitive merchant records, customer personal identifying information (PII), and financial history logs. These rules mandate strict constraints on the persistence layer: password hashes cannot be retrievable in plaintext, all session payloads must be signed using private asynchronous keys, and comprehensive database audit logs must capture every write modification.

2.4 Design Principles

To maintain high code quality and clear separation of concerns across both frontend and backend modules, the system architecture adheres to several industry-standard design principles.

SOLID Object-Oriented Design Principles

All classes, modules, and providers within the NestJS and Next.js applications follow the foundational SOLID principles:

- **Single Responsibility Principle (SRP):** Each class or service is responsible for exactly one business function (e.g., the ReceiptGenerationService handles formatting and streaming data, but does not process payments).
- **Open/Closed Principle (OCP):** Application modules are open for extension but closed for modification, utilizing abstract interfaces and dependency injection.
- **Liskov Substitution Principle (LSP):** Subtypes can substitute for their base types without altering system correctness.
- **Interface Segregation Principle (ISP):** Clients are not forced to depend on extensive interfaces they do not use; domain boundaries use concise, specialized contracts.
- **Dependency Inversion Principle (DIP):** High-level business logic modules do not import low-level data access layers directly; both depend on abstract contracts, implemented via NestJS’s dependency injection engine.

Don't Repeat Yourself (DRY)

The codebase avoids duplicate code structures by encapsulating cross-cutting workflows into reusable utility layers, custom middleware, and shared hooks. Common procedures, such as calculating product taxes, verifying permission grants, or formatting currency strings, are declared once within shared libraries and referenced uniformly across all domain modules.

Separation of Concerns (SoC)

The application architecture divides responsibilities across distinct operational layers:

```
Presentation Layer:  Next.js + Tailwind CSS
        |
        v   (REST API / JWT Auth)
Controller/Routing Layer:  Request Validation
        |
        v
Business Logic Layer:  NestJS Services & Domain Rules
        |
        v
Persistence Layer:  Prisma ORM + PostgreSQL 18
```

This strict layering prevents database entities from leaking into presentation views and ensures that user interface state alterations cannot bypass backend business validation rules.

2.5 Architectural Patterns

The complete system is built around standard, enterprise-proven architectural patterns that balance performance with modular maintainability.

Monolithic Modular Backend Architecture

The backend uses a modular monolithic architecture. Unlike decoupled microservices, which add significant network overhead and deployment complexity, all domains are compiled into a single executable runtime unit. However, logical modularity is strictly enforced: domains like UserModule, InventoryModule, and SalesModule maintain internal encapsulation, communicating exclusively via injected services. This design provides a clean migration path to an independent microservices architecture if transaction volumes eventually require physical service separation.

Model-View-Controller (MVC) and Layered Domain Architecture

The presentation and interaction models rely on a modernized variant of the layered MVC pattern. The Next.js frontend acts as an active presentation view that binds data fields to structural form models. On the server side, NestJS Controllers intercept incoming network payloads, delegate business operations to Domain Services, and interact with the database using Prisma ORM Data Models.

Repository and Data Mapper Patterns

To decouple business logic from raw SQL query execution, the database tier uses data mapper abstractions managed by Prisma ORM. Database schemas are compiled into strongly typed application objects. Complex querying operations that require advanced performance optimization are isolated within specialized repository classes, shielding the service layer from schema modification risks.

2.6 Technology Selection Rationale

Each component within the chosen technology stack was selected to satisfy specific operational requirements, development velocities, and performance targets.

BMS ADVANCED TECHNOLOGY STACK

FRONTEND TIER	BACKEND TIER
Next.js (App Router)	NestJS Framework
TypeScript	Prisma ORM
Tailwind CSS + Shadcn UI	TypeScript Runtime
PERSISTENCE, CACHING & INFRASTRUCTURE TIER	
PostgreSQL 18 (ACID Core)	Redis (Session & Cache)
Nginx (Reverse Proxy)	Docker & Docker Compose

Frontend Framework: Next.js, Tailwind CSS, and Shadcn UI

Next.js provides an optimal frontend architecture by merging server-side rendering performance with reactive client-side interfaces. Server Components fetch initial layouts directly within the cloud infrastructure, significantly speeding up First Contentful Paint metrics. Tailwind CSS ensures low asset sizes by generating atomic utility styles, avoiding the bloat of traditional UI frameworks. Shadcn UI provides unstyled, accessible primitives that give engineers complete control over DOM behavior, which is essential for building a fast, accessible Point of Sale interface.

Backend Engine: NestJS and TypeScript

NestJS provides a highly structured engineering architecture for Node.js environments. By enforcing TypeScript development out of the box, it ensures complete type safety from backend database access points all the way to API contracts. NestJS's powerful modular dependency injection system simplifies testing and makes it easy to replace mock implementations with real services. This allows teams to build highly maintainable codebases that can scale alongside growing business logic complexity.

Persistence Engine: PostgreSQL 18 and Prisma ORM

PostgreSQL 18 offers advanced relational capabilities, reliable ACID compliance, and excellent connection handling under heavy concurrent write loads. Prisma ORM translates these capabilities into the application space by auto-generating TypeScript interfaces directly from the database schema definition. This setup catches structural type mismatches during compilation rather than at runtime, while still allowing developers to write optimized, high-performance raw SQL queries for complex analytical reporting when needed.

Performance Caching & Infrastructure: Redis, Nginx, and Docker

Redis acts as an in-memory database that handles high-throughput session cache states, short-lived API responses, and rate-limiting counters, offloading repetitive query stress from the relational database. Nginx sits at the edge of the architecture, serving as a high-performance reverse proxy that manages TLS/SSL termination, applies security headers, and decompresses static client assets. Finally, Docker and Docker Compose wrap every service into consistent, isolated software

containers, ensuring predictable behavior and identical runtime environments from local development all the way to production clusters.

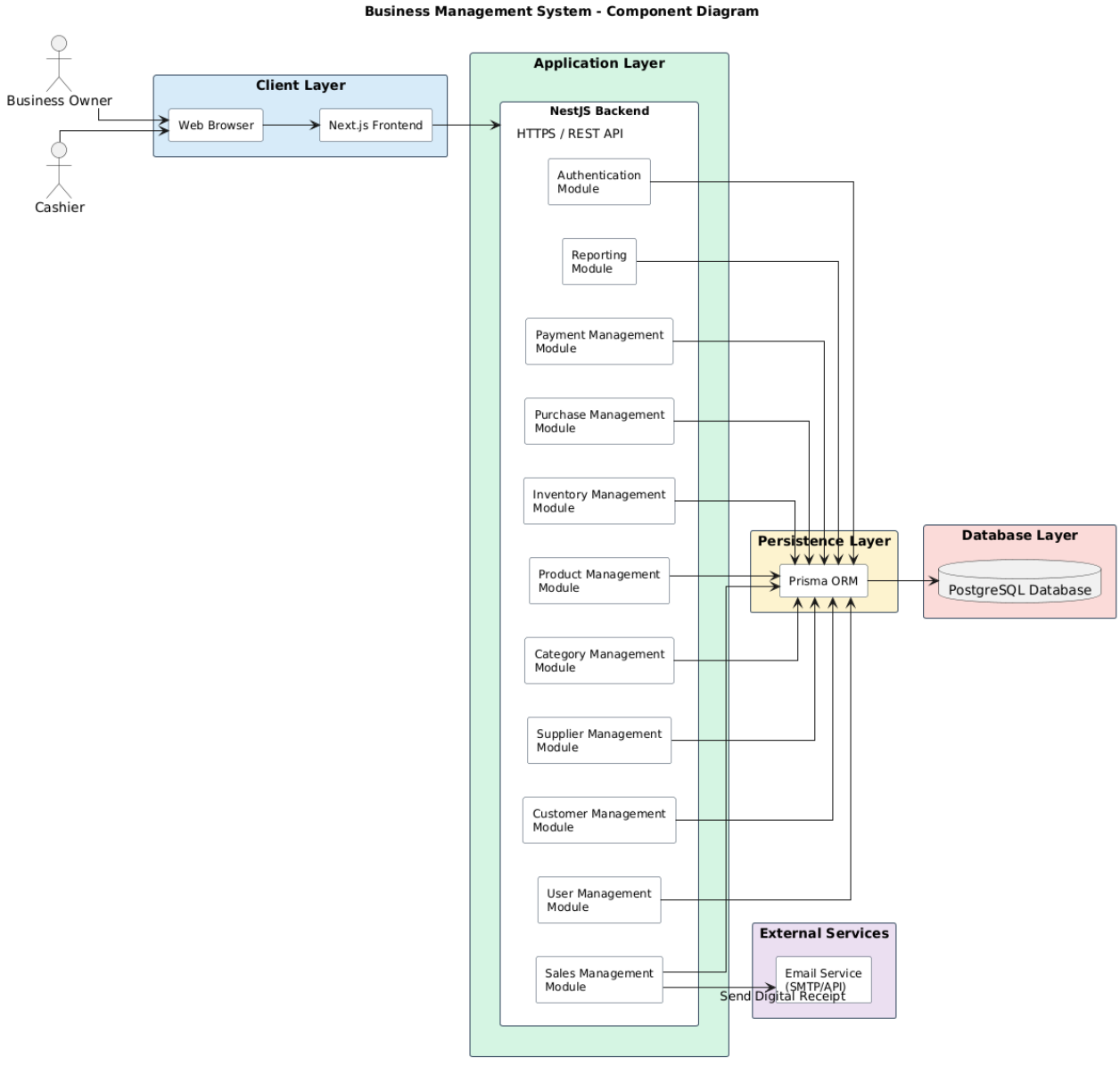
3. SYSTEM ARCHITECTURE

This section details the static structural configuration, component relationships, physical deployment topology, and modular dependency organization of the Business Management System (BMS).

3.1 Static Component Sub-systems

The runtime engine of the BMS is decomposed into distinct operational components designed to enforce a strict separation of concerns, optimize horizontal scalability, and decouple user interaction layers from transactional data consistency mechanisms.

Figure 3.1 – Component Diagram



Component Dissection and Interaction Mechanics

The system architecture consists of four distinct operational layers interacting over contract-governed interfaces:

1. Presentation Client Component (Next.js Application Stack)

The front-end presentation layer functions as an independent, optimized visual client executing within the end-user's web runtime environment. Built on Next.js, this component is structurally segregated into:

- **Static Layout Engine:** Uses Server Components to fetch initial metadata states directly inside the deployment infrastructure, significantly decreasing time-to-interactive metrics.
- **Reactive State Engine (POS Client Workspace):** Operates using Client Components to provide low-latency, immediate visual feedback during cart modifications, barcode interpretation, and cash register operations.
- **API Client Layer:** A structured interface wrapper utilizing Axios or the native fetch API to route authenticated asynchronous requests across TLS-encrypted boundaries.

2. Network Routing & Reverse Proxy Gateway Component (Nginx Engine)

Nginx sits at the edge of the computing cluster, acting as a high-performance reverse proxy and load balancer. Its explicit responsibilities include:

- **TLS/SSL Termination:** Offloads cryptographic computation overhead from the Node.js application process by resolving certificates at the edge.
- **Static Asset Offloading:** Directly intercepts and serves serialized client bundles, visual media resources, and compiled stylesheets without forwarding requests to the application containers.
- **Rate Limiting & Threat Shielding:** Appends low-level volumetric protection constraints on a per-IP basis, preventing denial-of-service attempts.

3. Core Application Service Component (NestJS Framework Engine)

The centralized application intelligence engine is a modular monolith constructed on the NestJS framework. It processes client payloads through a sequential, deterministic pipeline:

- **HTTP Controller Pipeline:** Intercepts REST network requests, unmarshalls incoming serialized payloads, and applies input verification constraints using declarative class validators.
- **Guard & Interceptor Middlewares:** Executes identity validation by resolving incoming JSON Web Tokens (JWT) against shared keys, and implements Role-Based Access Control (RBAC) matrices before routing to business services.
- **Domain Service Tier:** Contains the core business logic of the enterprise, coordinating state transitions, computing transaction ledger balances, and generating digital analytical representations.
- **Data Access Layer (Prisma ORM):** Abstracts database engines by translating business models into structured PostgreSQL relational expressions.

4. Shared Caching & Persistence Sub-systems

The state storage engine relies on two distinct data engines:

- **In-Memory Volatile Cache (Redis Cluster):** Stores short-lived active JWT identity structures, computational rate-limit tables, and read-heavy inventory product definitions to bypass database querying bottlenecks.
- **Persistent Relational Database Core (PostgreSQL 18):** Serves as the absolute single source of truth for the enterprise. It handles durable multi-tenant transactional storage under strict ACID parameters.

3.2 Production Deployment Infrastructure

The production infrastructure architecture is designed for containerized isolation, deterministic continuous deployment, high horizontal availability, and absolute perimeter security.

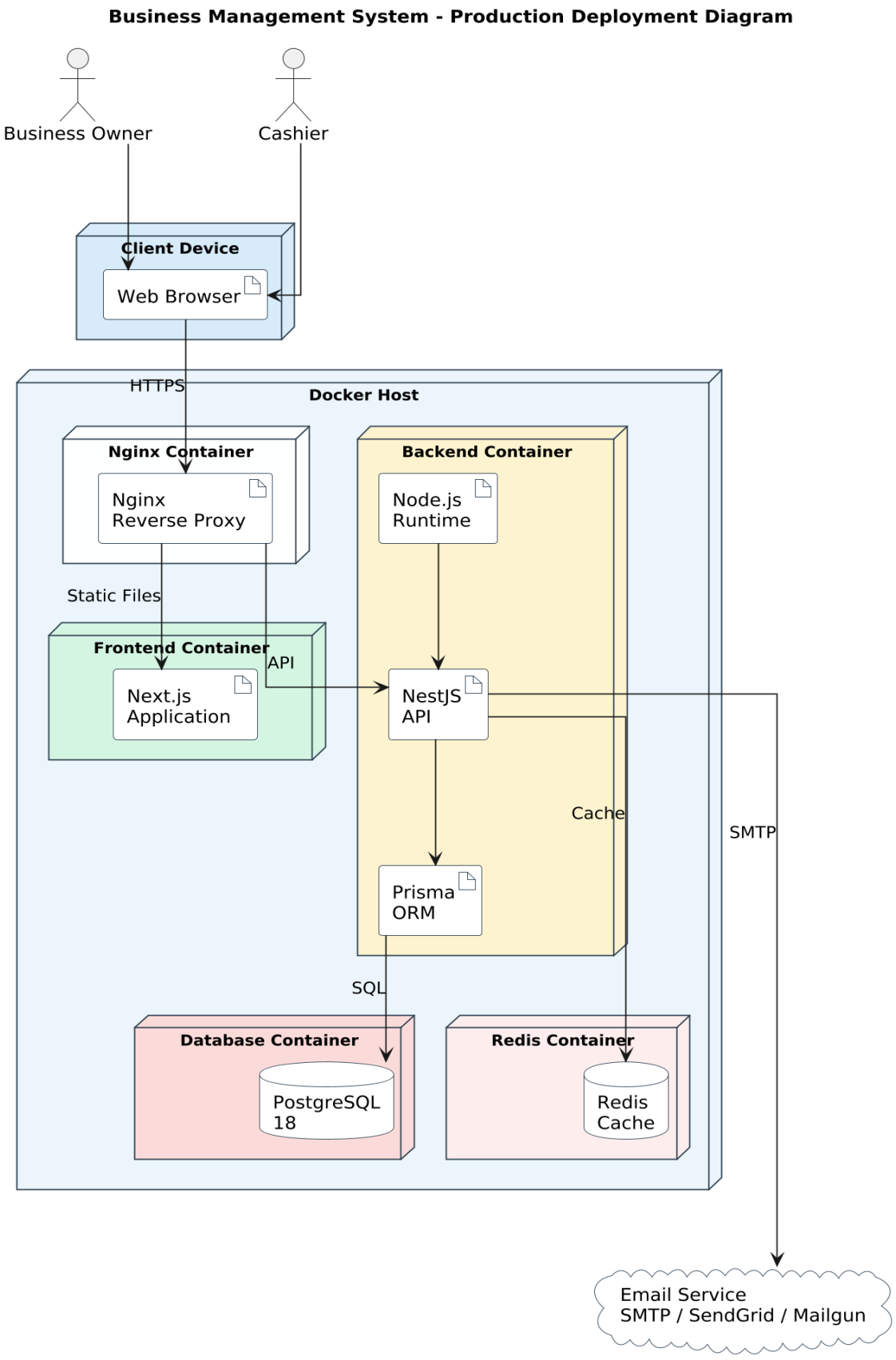


Figure 3.2 – Deployment Diagram

Infrastructure Tier Breakdowns and Network Topologies

The runtime topology is physically split across three security-isolated sub-networks within the cloud provisioning zone:

1. Public Perimeter Network Tier (DMZ Edge Network)

The outermost operational layer contains the public cloud load balancers and the edge Nginx reverse proxy routing nodes. This tier maps inbound standard web traffic (Ports 80 and 443) to internal application containers. No database components or business logic engines are hosted within this public execution space.

2. Private Application Tier (VPC Protected Sub-net)

The core application code runs inside isolated Linux Docker containers managed by Docker Compose or an orchestrator within a secure Virtual Private Cloud (VPC). The NestJS service nodes run in a private network segment, communicating exclusively over port 3000 with the Nginx edge proxy. This layout allows for rapid horizontal scaling; additional instances can be spun up on demand behind internal private round-robin routing models.

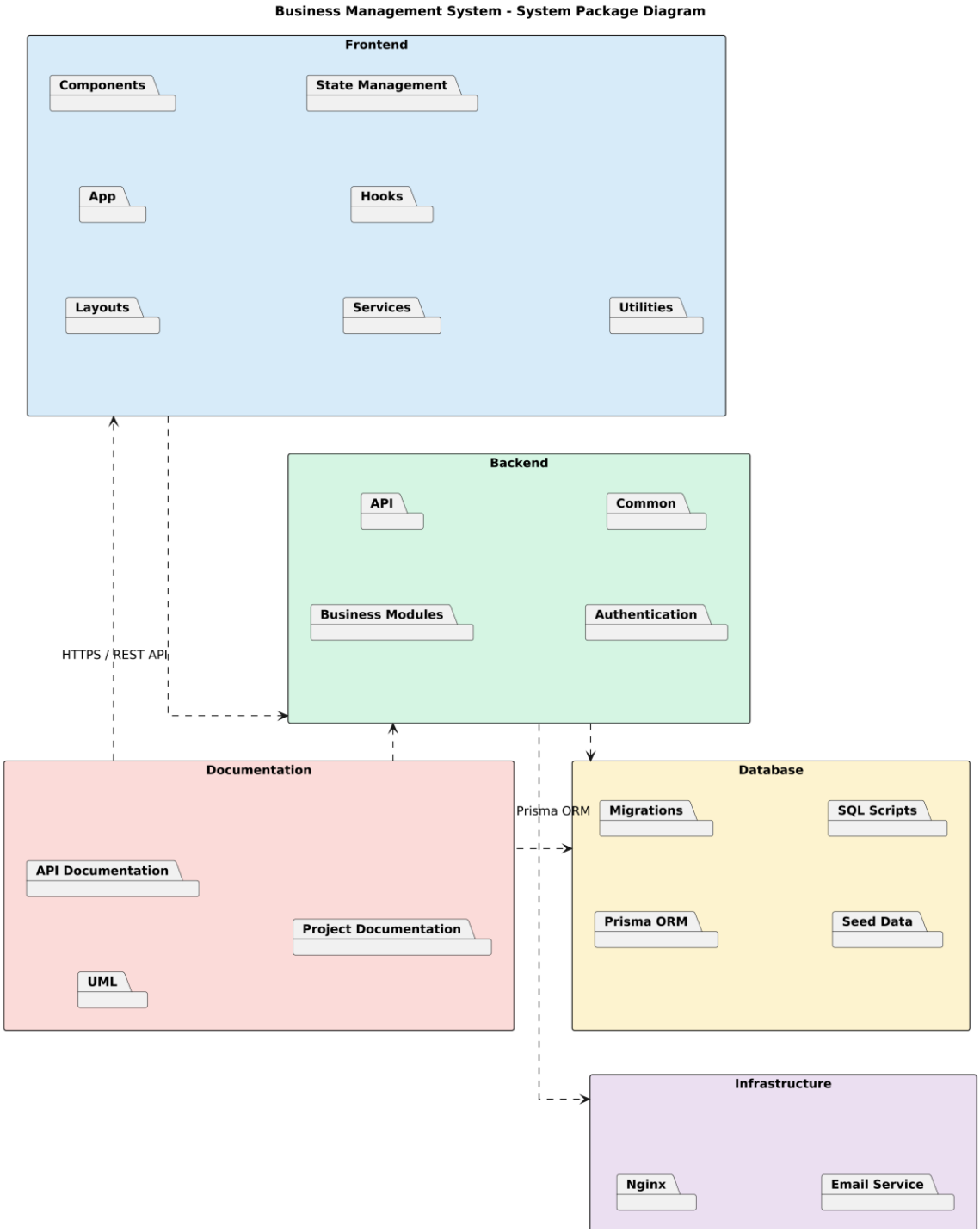
3. Secure Persistence Tier (Deep Private Database Isolation Sub-net)

The data layer is isolated within the deepest private tier of the VPC, completely hidden from direct external internet visibility. The PostgreSQL 18 relational instance accepts incoming TCP connections exclusively on Port 5432 from verified application container IPs. Redis is similarly configured inside this private tier on Port 6379, preventing any unauthorized external exposure of session states or volatile application data caches.

3.3 System Package Organization

The structural division of the software codebase across repositories and file boundaries uses a strict package system to ensure uniform code distribution and prevent dependency loops.

Figure 3.3 – System Package Diagram



Package Architecture Definitions

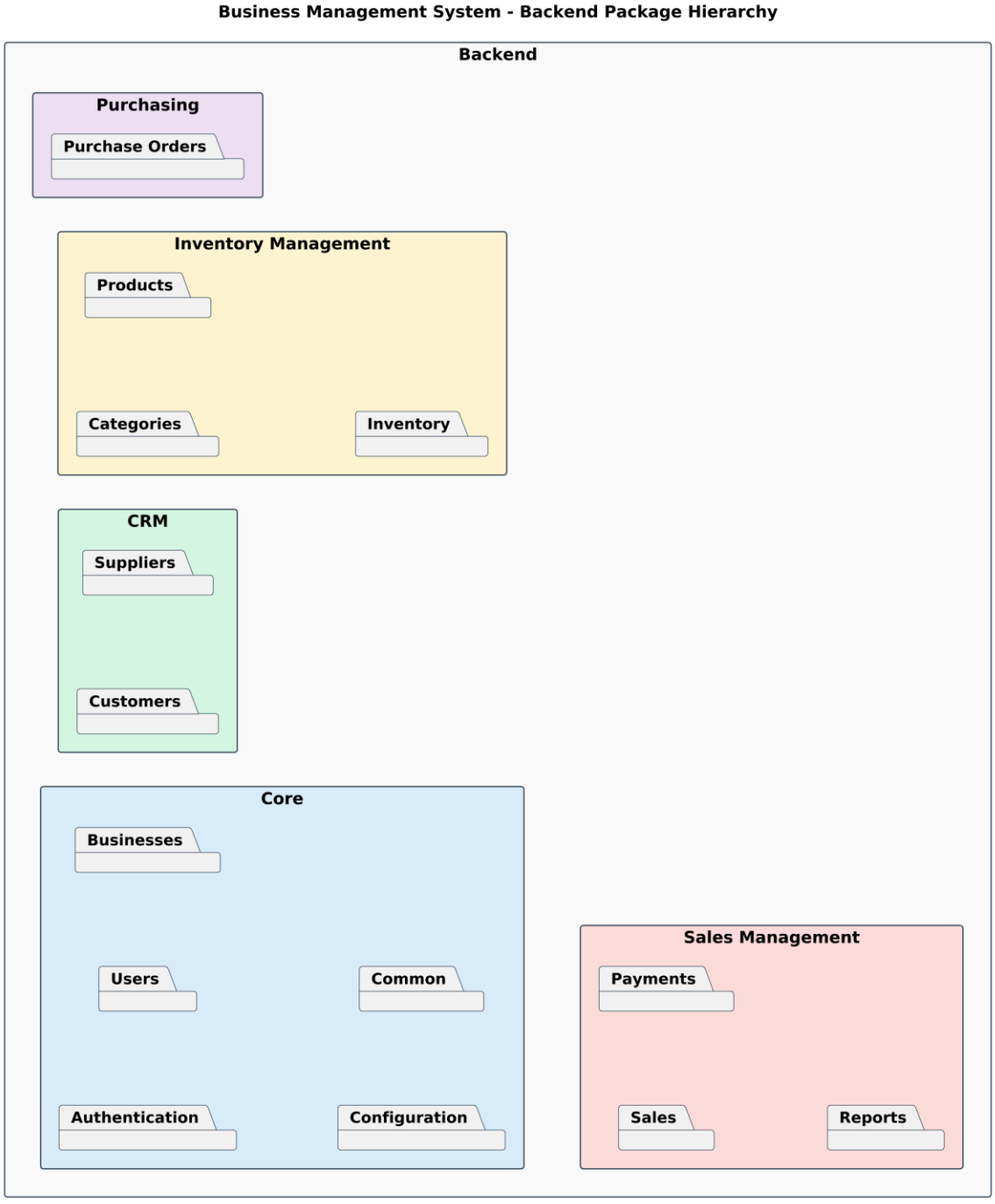
The codebase is structured into three parent architectural packages:

- **apps/bms-frontend (Presentation Package):** A structured layout including /components (reusable UI blocks), /hooks (state synchronization), /services (API layer), and /app (Next.js file-system routing nodes). This package depends entirely on the public API definitions exposed by the backend package.
- **apps/bms-backend (Application Core Package):** A modular system structured into independent domain modules containing controllers, services, entities, and data transfer object (DTO) contracts.
- **libs/bms-shared-contracts (Shared Type Specification Package):** A localized internal package containing raw TypeScript type mappings, data schema validation matrices, and shared error exception classes used by both frontend views and backend services to ensure complete compile-time type safety.

3.4 Backend Package Domain Hierarchy

The backend application code is organized into isolated domain boundaries inside the NestJS ecosystem. Each module manages a specific area of enterprise business operations.

Figure 3.4 – Backend Package Hierarchy



Functional Domain Module Classifications

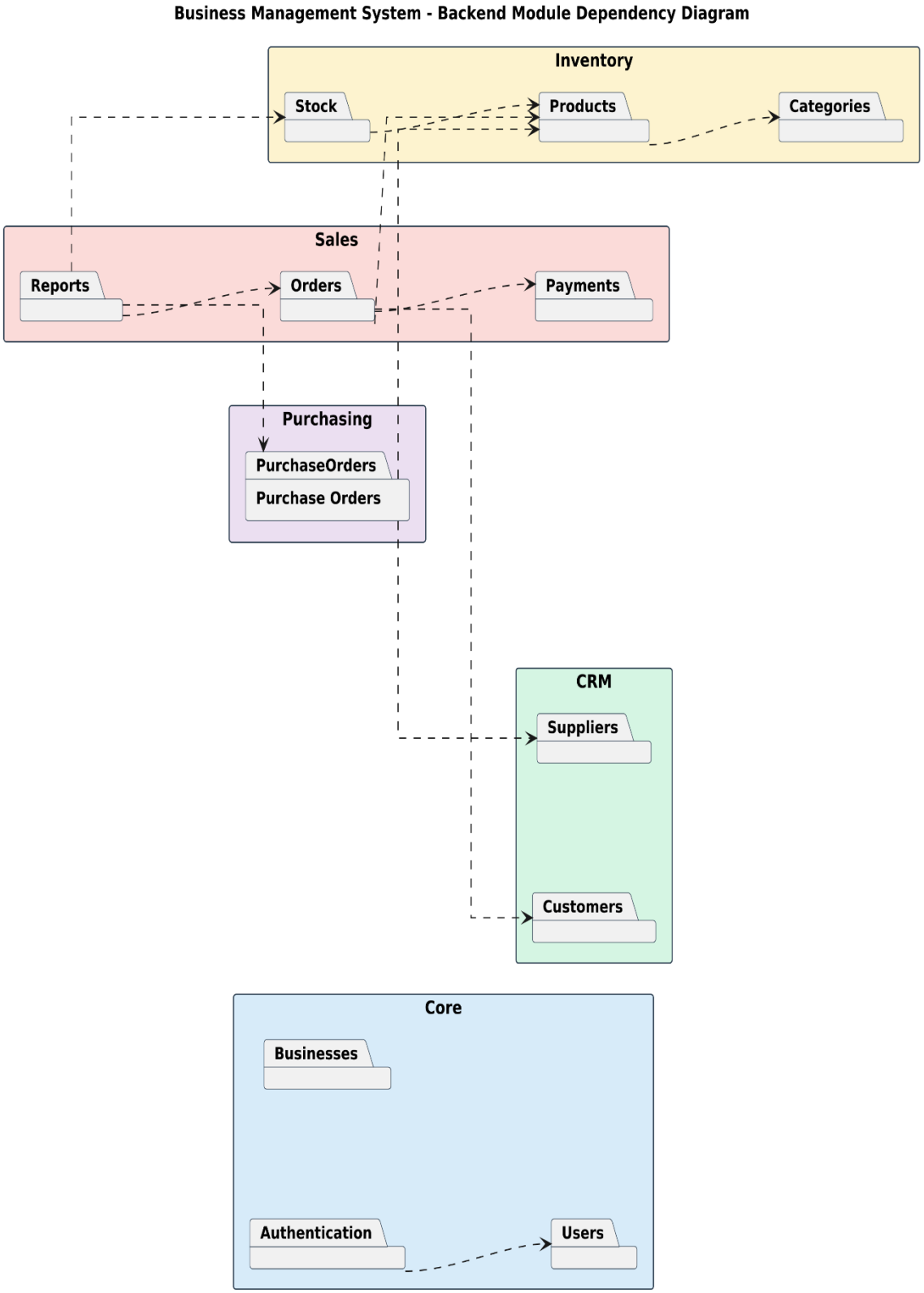
- **AuthModule & UserModule (Identity Context):** Manages user registration profiles, password security operations, JWT validation routines, and the resolution of RBAC permission scopes.

- **BusinessModule (Tenant Context):** Acts as the parent container for multi-tenant profiles, defining corporate tax identification rules, currency preferences, and localized organizational structural parameters.
- **InventoryModule & ProductModule (Catalog Context):** Governs structural parameters for items, global pricing classifications, categories, real-time inventory levels, and stock adjustment trails.
- **SalesModule & PaymentModule (Transaction Context):** Manages checkout mechanics, computes promotional deductions, updates item numbers, coordinates third-party payment handshakes, and generates digital receipt structures.
- **SupplyChainModule (Procurement Context):** Houses supplier management logic, tracks outstanding purchase orders, and manages receipt records when incoming stock arrives at a facility.
- **AnalyticsModule (Reporting Context):** Pulls from specialized read-views to provide aggregations for sales performance data, profit calculations, and historical stock trends.

3.5 Backend Module Dependency Graph

To ensure long-term maintainability and prevent circular dependency locks, individual domain modules must interact through structured dependency injection paths.

Figure 3.5 – Backend Module Dependency Diagram



Architectural Rules for Dependency Distribution

The system follows a strict hierarchical dependency graph to prevent architectural coupling. At the foundation sits the PrismaModule (persistence injection), which provides database access across the entire stack. Above the database tier are core infrastructure modules like AuthModule and UserModule.

Domain modules — such as ProductModule and InventoryModule — import the core persistence layer but operate independently from each other. At the top of the graph sits the SalesModule, which orchestrates complex operations across multiple domains. It imports dependencies from the InventoryModule (to adjust stock counts), the UserModule (to assign staff accountability traits), and the PaymentModule (to settle transactional totals).

Circular references (e.g., SalesModule importing InventoryModule while InventoryModule attempts to import SalesModule directly) are forbidden at the compiler level. Any cross-domain communication that cannot be handled via direct service injection must use event-driven messaging or specialized intermediate application services.

4. DATABASE DESIGN

The data persistence tier of the Business Management System (BMS) is engineered on the PostgreSQL 18 relational database engine. The schema design models multi-tenant enterprise data structures, supporting high-throughput transaction processing, historical inventory traceability, and strict financial data consistency.

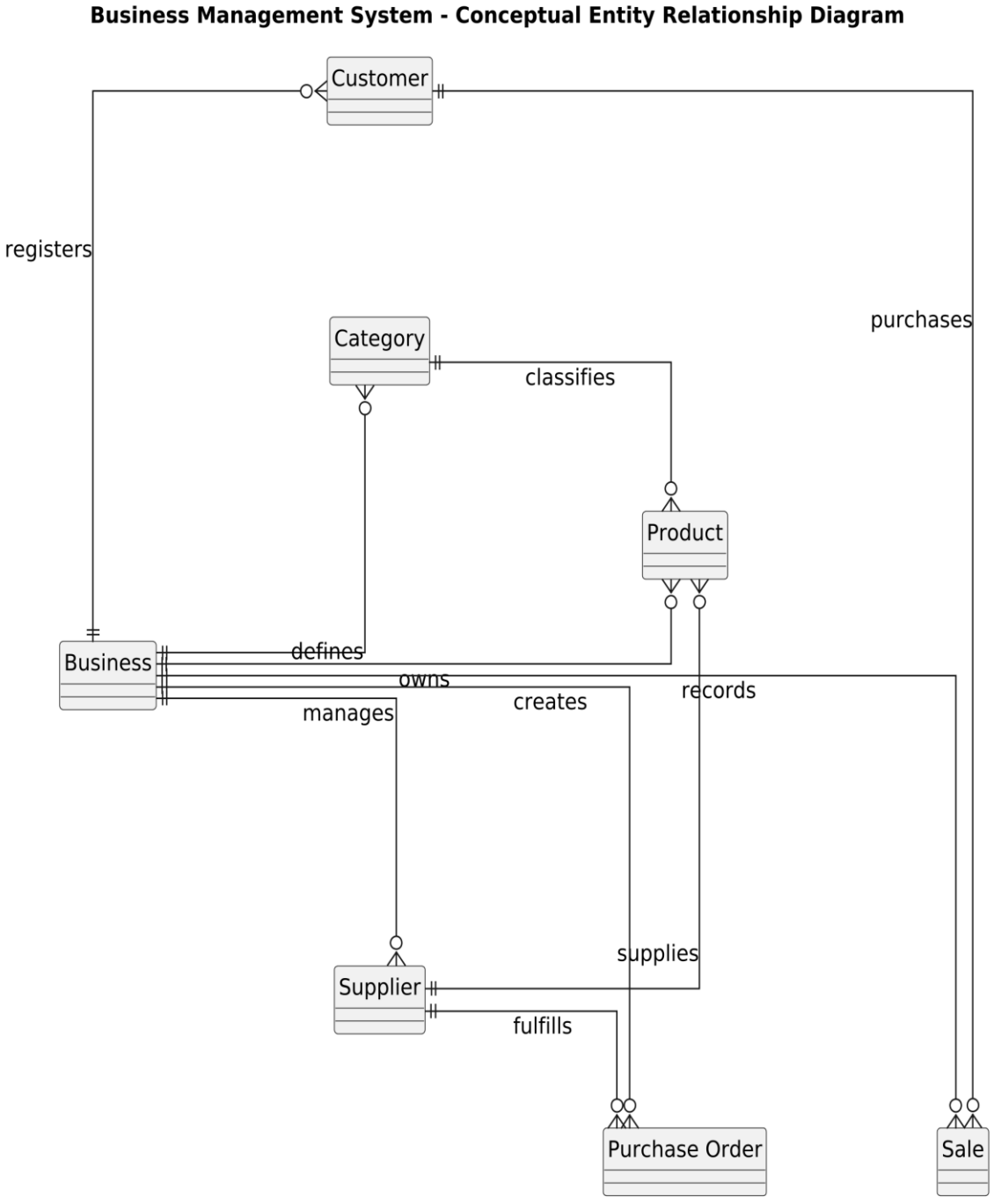
4.1 Relational Architecture Models

4.1.1 Conceptual Data Framework

The high-level conceptual model defines the operational layout of the system, establishing clear boundaries for multi-tenant isolation, supply chains, workforce tracking, and point-of-sale checkout workflows.

The conceptual model establishes a root enterprise entity that owns all operational domains. Tenants are logically isolated: a central corporate entity controls independent groupings of inventory items, supplier catalogs, human operator profiles, and customer directories. Sales transactions and incoming supply chains link directly to this corporate context, ensuring that cross-tenant data leaks are prevented at the relational boundary.

Figure 4.1 – Conceptual ERD



4.1.2 Logical Schema Model

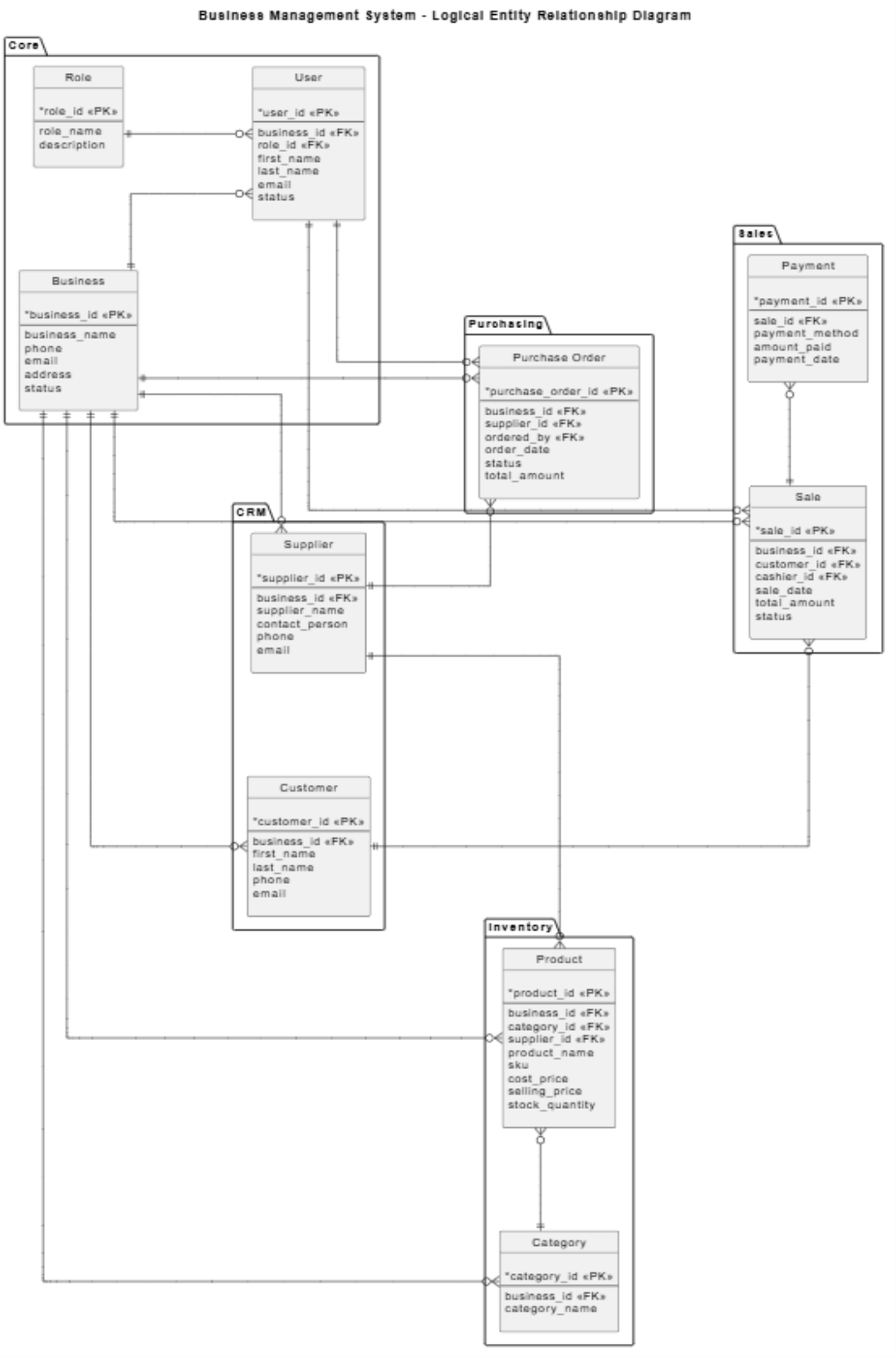
The logical representation maps conceptual domains into structured relational entities, defining strict data types, entity constraints, nullability parameters, and structural relationships.

The logical system uses a strictly normalized structure to remove relational redundancy and maintain high database integrity:

- businesses retain a one-to-many relationship with users, products, customers, suppliers, and sales.
- categories form a hierarchical tree pattern linked to products.
- products maintain a one-to-many relationship with inventory_items, tracking localized stock quantities across different physical branches.
- sales connect one-to-many with sale_items, which reference specific products.
- purchase_orders map a one-to-many relationship to purchase_order_items, linking procurement workflows directly to suppliers and products.

Figure 4.2 – Logical ERD

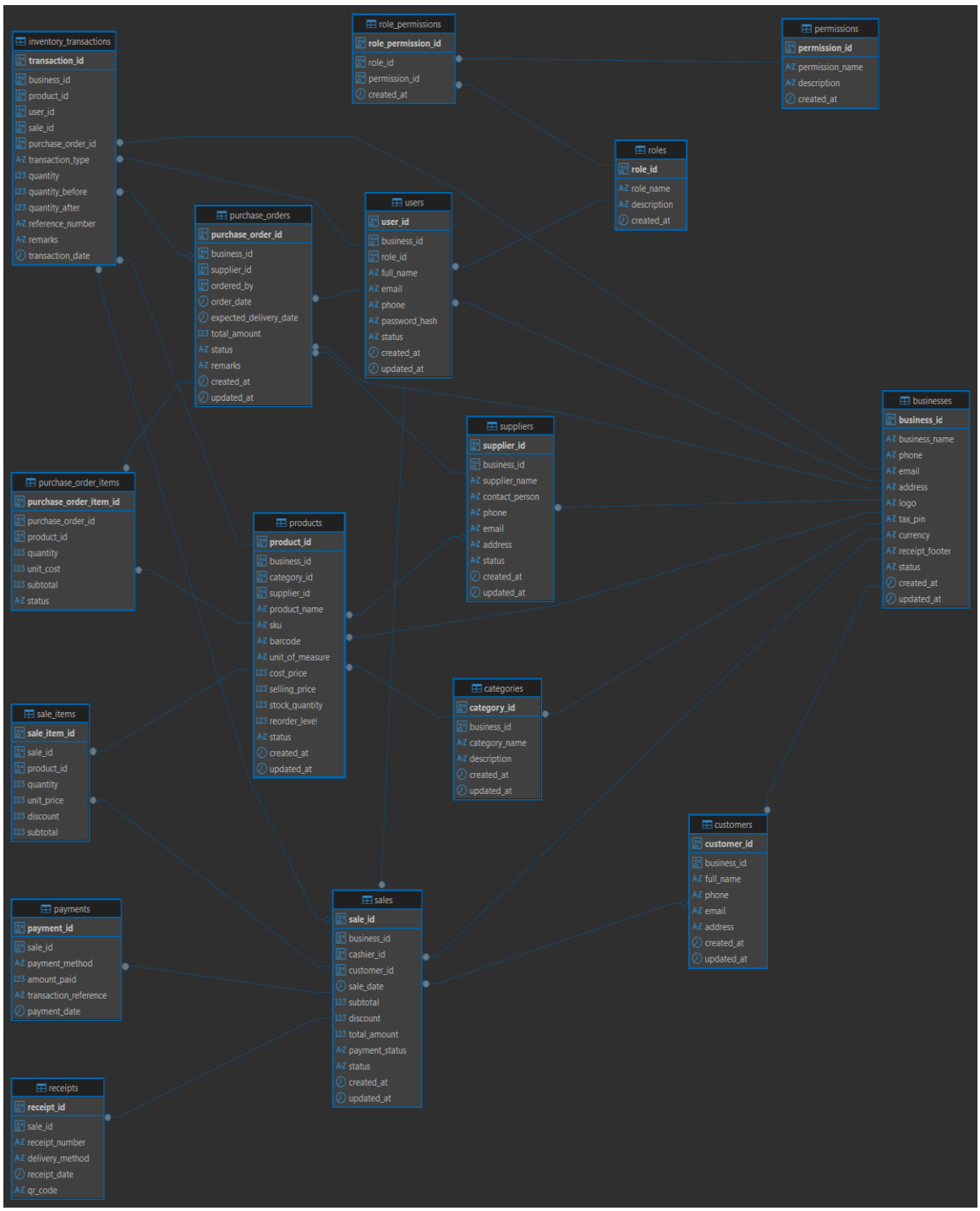
- .



4.1.3 Physical Storage Model

The physical design details how the data structures are implemented within the PostgreSQL 18 engine, configuring optimized column data types, precision parameters, foreign key cascades, and low-level data storage engines.

Figure 4.3 – Physical ERD



Physical columns use explicit data type definitions to maximize storage efficiency and preserve mathematical accuracy. Identifiers are mapped as globally unique UUIDv4 strings to prevent sequential discovery attacks and simplify multi-region synchronization. Text-heavy fields use the standard VARCHAR type with explicitly defined length limits, while large JSON fields use the optimized binary jsonb format.

4.2 Database Schema Specifications

4.2.1 Core Relational Tables

The database schema consists of twelve core structural tables. Below are the precise data definitions and structural constraints for each table:

1. businesses

Stores the high-level corporate configurations, tax identifiers, and regional settings for each tenant.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`name: VARCHAR(255)`	Not Null
`tax_number: VARCHAR(50)`	Nullable | Unique within business context
`currency: VARCHAR(3)`	Not Null | DEFAULT 'USD'
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP
`updated_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

2. users

Contains the workforce credentials, identity states, and role allocations.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`email: VARCHAR(255)`	Not Null | Global Unique Constraint
`password_hash: VARCHAR(255)`	Not Null
`role: VARCHAR(50)`	Not Null | Constraint (ADMIN, MANAGER, CASHIER)
`is_active: BOOLEAN`	Not Null | DEFAULT TRUE
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

3. categories

Provides hierarchical grouping structures for the product catalog.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade

Column	Definition
`parent_id: UUID`	Foreign Key (categories.id) | Nullable | Self-Referential
`name: VARCHAR(100)`	Not Null
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

4. products

Defines the individual items, pricing tiers, and identification barcodes.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`category_id: UUID`	Foreign Key (categories.id) | On Delete Set Null
`sku: VARCHAR(100)`	Not Null | Unique per Business Context
`barcode: VARCHAR(100)`	Nullable
`name: VARCHAR(255)`	Not Null
`price: NUMERIC(12, 2)`	Not Null | Check (price >= 0.00)
`cost: NUMERIC(12, 2)`	Not Null | Check (cost >= 0.00)
`is_tracked: BOOLEAN`	Not Null | DEFAULT TRUE
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

5. inventory_items

Tracks the current physical stock numbers for tracked items.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`product_id: UUID`	Foreign Key (products.id) | On Delete Cascade | Unique
`quantity: INT`	Not Null | DEFAULT 0
`low_stock_threshold: INT`	Not Null | DEFAULT 10
`updated_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

6. customers

Maintains demographic records and trade accounts for consumer transactions.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`name: VARCHAR(255)`	Not Null

Column	Definition
`email: VARCHAR(255)`	Nullable
`phone: VARCHAR(50)`	Nullable
`balance: NUMERIC(12, 2)`	Not Null | DEFAULT 0.00

7. suppliers

Tracks distributor coordinates, order histories, and ledger liabilities.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`company_name: VARCHAR(255)`	Not Null
`contact_name: VARCHAR(150)`	Nullable
`phone: VARCHAR(50)`	Nullable
`balance: NUMERIC(12, 2)`	Not Null | DEFAULT 0.00

8. purchase_orders

Manages B2B procurement histories and physical incoming inventory deliveries.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`supplier_id: UUID`	Foreign Key (suppliers.id) | On Delete Restrict
`status: VARCHAR(50)`	Not Null | Constraint (DRAFT, SENT, RECEIVED, CANCELLED)
`total_amount: NUMERIC(12, 2)`	Not Null | DEFAULT 0.00
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

9. purchase_order_items

Line item records mapping requested product quantities inside a purchase order.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`purchase_order_id: UUID`	Foreign Key (purchase_orders.id) | On Delete Cascade
`product_id: UUID`	Foreign Key (products.id) | On Delete Restrict
`quantity_ordered: INT`	Not Null | Check (quantity_ordered > 0)
`quantity_received: INT`	Not Null | DEFAULT 0
`unit_cost: NUMERIC(12, 2)`	Not Null

10. sales

The transactional header capturing completed consumer checkouts.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`business_id: UUID`	Foreign Key (businesses.id) | On Delete Cascade
`user_id: UUID`	Foreign Key (users.id) | On Delete Restrict
`customer_id: UUID`	Foreign Key (customers.id) | Nullable | On Delete Restrict
`invoice_number: VARCHAR(100)`	Not Null | Unique per Business Context
`subtotal: NUMERIC(12, 2)`	Not Null
`tax_amount: NUMERIC(12, 2)`	Not Null
`discount_amount: NUMERIC(12, 2)`	Not Null | DEFAULT 0.00
`total_amount: NUMERIC(12, 2)`	Not Null
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

11. sale_items

Individual transaction lines capturing specific quantities and snapshots of product values.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`sale_id: UUID`	Foreign Key (sales.id) | On Delete Cascade
`product_id: UUID`	Foreign Key (products.id) | On Delete Restrict
`quantity: INT`	Not Null | Check (quantity > 0)
`unit_price: NUMERIC(12, 2)`	Not Null
`total_price: NUMERIC(12, 2)`	Not Null

12. payments

Tracks checkout settlements and transactional payment clearings.

Column	Definition
`id: UUID`	Primary Key | DEFAULT gen_random_uuid()
`sale_id: UUID`	Foreign Key (sales.id) | On Delete Cascade
`method: VARCHAR(50)`	Not Null | Constraint (CASH, CARD, MOBILE_MONEY)
`amount: NUMERIC(12, 2)`	Not Null
`transaction_reference: VARCHAR(150)`	Nullable
`created_at: TIMESTAMP`	Not Null | DEFAULT CURRENT_TIMESTAMP

4.3 Primary and Foreign Key Policies

All primary identifiers use non-sequential UUIDv4 keys to shield internal transactional volumes from external discovery. Foreign keys enforce referential boundaries using strict deletion strategies:

- **ON DELETE CASCADE:** Applied to tenant metadata groupings (such as users, products, and sales linked to a business_id). If an enterprise account is deleted, all dependent records are purged automatically.
- **ON DELETE RESTRICT:** Applied to critical historical ledgers (such as products linked to sale_items, or suppliers linked to purchase_orders). This policy blocks data deletions that would break historical audit trails.
- **ON DELETE SET NULL:** Applied to optional classification metadata (such as parent category mappings), keeping the primary product record intact if a category is deleted.

4.4 Advanced Constraints and Field Validations

To guarantee data consistency before application validations run, the database layer enforces native field-level constraints:

- **CHECK (price >= 0) and CHECK (cost >= 0):** Enforces positive values for product pricing and procurement costs across the system.
- **CHECK (quantity >= 0):** Prevents stock values inside the inventory_items table from dropping below zero during high-concurrency checkouts.
- **Composite Scope Constraints:** Unique constraints combine the target field with the tenant identifier (e.g., UNIQUE(business_id, sku) and UNIQUE(business_id, invoice_number)). This configuration allows identical standard SKUs or invoice sequences to exist safely across different tenant environments without collisions.

4.5 Indexing Optimization Layouts

The database uses targeted database indexing to maintain low-latency query performance as transaction volumes scale:

- **idx_users_email:** A unique B-Tree index on the user email column to optimize authentication lookups.
- **idx_products_sku_barcode:** A multi-column index tracking both SKU and barcode fields, ensuring near-instantaneous item lookups during front-end POS scanning operations.
- **idx_inventory_product:** A covering index over product allocations that speeds up stock-level calculations during checkout.
- **idx_sales_business_date:** A composite index tracking (business_id, created_at DESC) that allows the analytics dashboard to quickly aggregate data without performing slow full-table scans.

4.6 Business Logic Triggers and Automated Procedures

The persistence layer uses database triggers to automate state updates and maintain absolute audit trails for tracking inventory.

4.6.1 Automated Updated Timestamp Automation

Every time an operational record is modified, a trigger executes to automatically update the updated_at column:

```
CREATE OR REPLACE FUNCTION set_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_businesses_updated_at
BEFORE UPDATE ON businesses
FOR EACH ROW EXECUTE FUNCTION set_updated_at_column();
```

4.6.2 Asynchronous Inventory Level Adjustments

When a sales transaction is finalized, a database trigger automatically deducts the purchased product quantities from the active inventory records:

```
CREATE OR REPLACE FUNCTION deduct_inventory_on_sale()
RETURNS TRIGGER AS $$
BEGIN
  IF EXISTS (SELECT 1 FROM products WHERE id = NEW.product_id AND is_tracked
= TRUE) THEN
    UPDATE inventory_items
    SET quantity = quantity - NEW.quantity
    WHERE product_id = NEW.product_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sale_items_insert
AFTER INSERT ON sale_items
FOR EACH ROW EXECUTE FUNCTION deduct_inventory_on_sale();
```

4.7 Database Seed Architecture

The seed configuration establishes the initial system roles, default entities, and administrator accounts required to bootstrap a new environment.

The seeding routine inserts base system access profiles (ADMIN, MANAGER, CASHIER) into user identity states, registers a default cash customer profile (Walk-in Customer) for every new business account, and seeds standard operational taxonomy nodes to ensure the system is ready for immediate deployment.

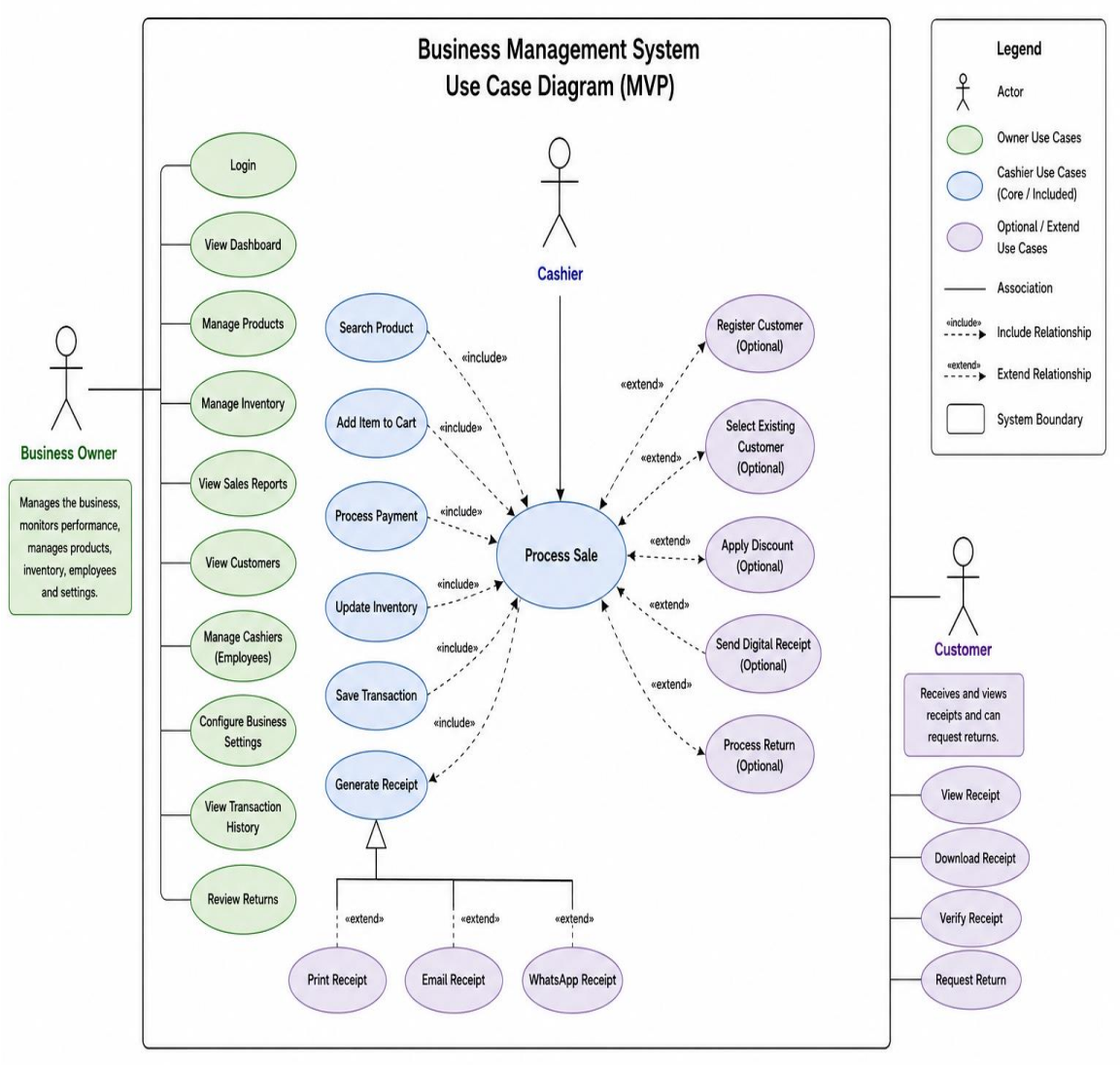
5. BEHAVIOURAL DESIGN

This chapter describes the dynamic runtime behavior, system interactions, and sequential states of the Business Management System (BMS). It details how system actors interact with software components to execute business processes.

5.1 Use Case Diagram

The Minimum Viable Product (MVP) system boundary encompasses three primary user actors interacting with core business modules.

Figure 5.1 – Use Case Diagram



System Actors and Boundary Specifications

- **Cashier:** The transactional frontline operator. Grounded within the restricted runtime context of the POS sub-system, this actor is authorized to scan product barcodes, manage active sales carts, execute transactional checkout requests, accept settlements, and trigger digital receipts.
- **Manager:** The operational supervisor. This actor inherits full lower-level Cashier transactional execution capabilities while possessing exclusive authorization to modify product price matrices, manage supplier directories, establish inventory restock parameters, and generate purchase orders.
- **Admin (System Administrator):** The root governance actor. Responsible for establishing multi-tenant corporate records, provisioning system user accounts, modifying high-level Role-Based Access Control (RBAC) permission vectors, and overriding critical financial records.

5.2 Activity Diagrams

5.2.1 Login Activity

- **Purpose:** To securely verify user identity credentials and issue a valid cryptographic session before granting system access.
- **Description:** Intercepts client authentication requests, executes secure password decryption-resistant verification routines, and handles both successful access grants and multi-tier authentication failures.

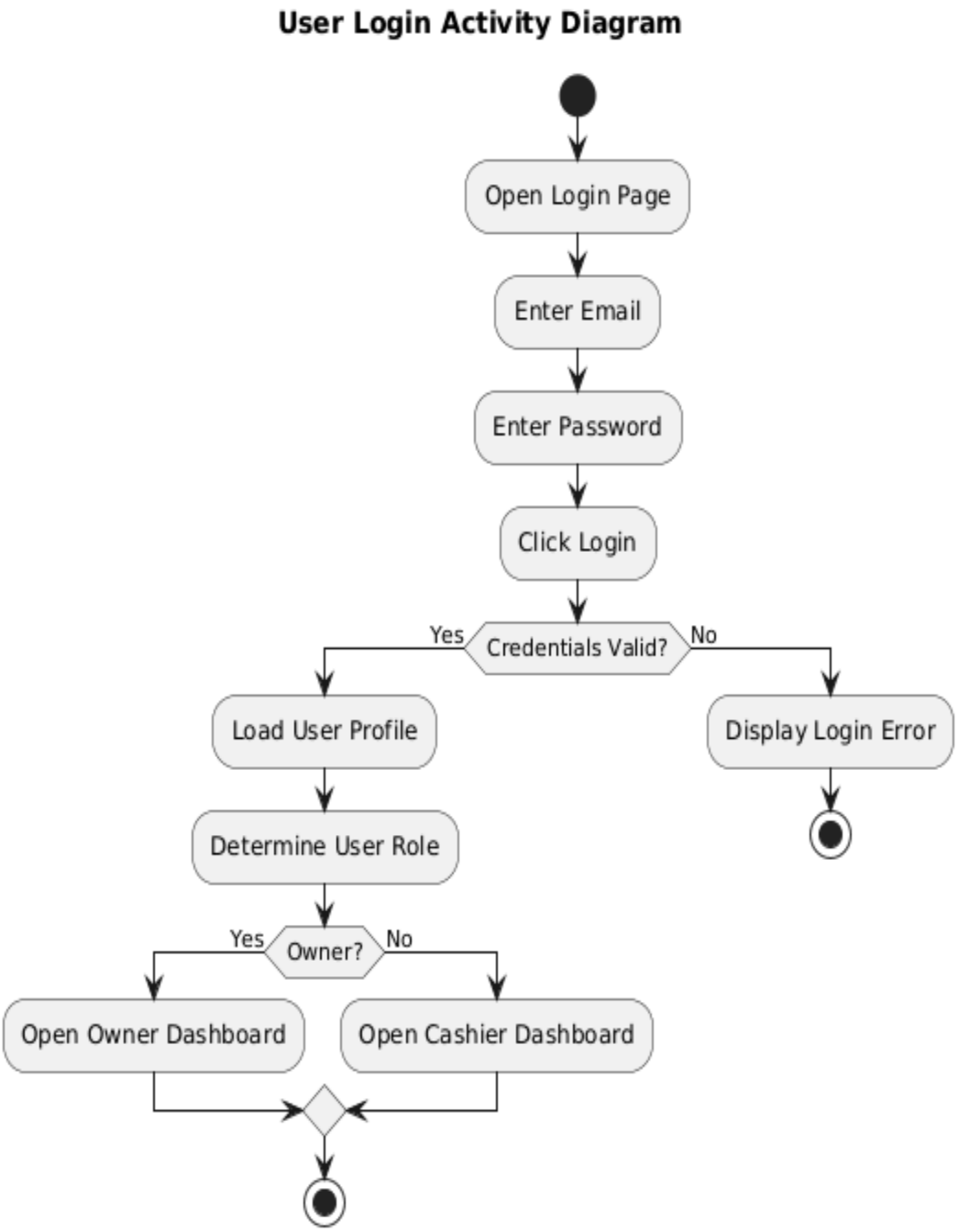
Workflow Explanation:

The user submits their email address and plaintext password via the Next.js login view. The frontend client serializes this payload and sends an HTTP POST request to the backend /auth/login gateway.

The server runs input validation checks; if the formatting fails, a 400 Bad Request response is immediately returned. If the formatting is valid, the server reads the corresponding user record from the database. The system then evaluates the encoded password hash against the database value using an encryption-resistant hashing pipeline.

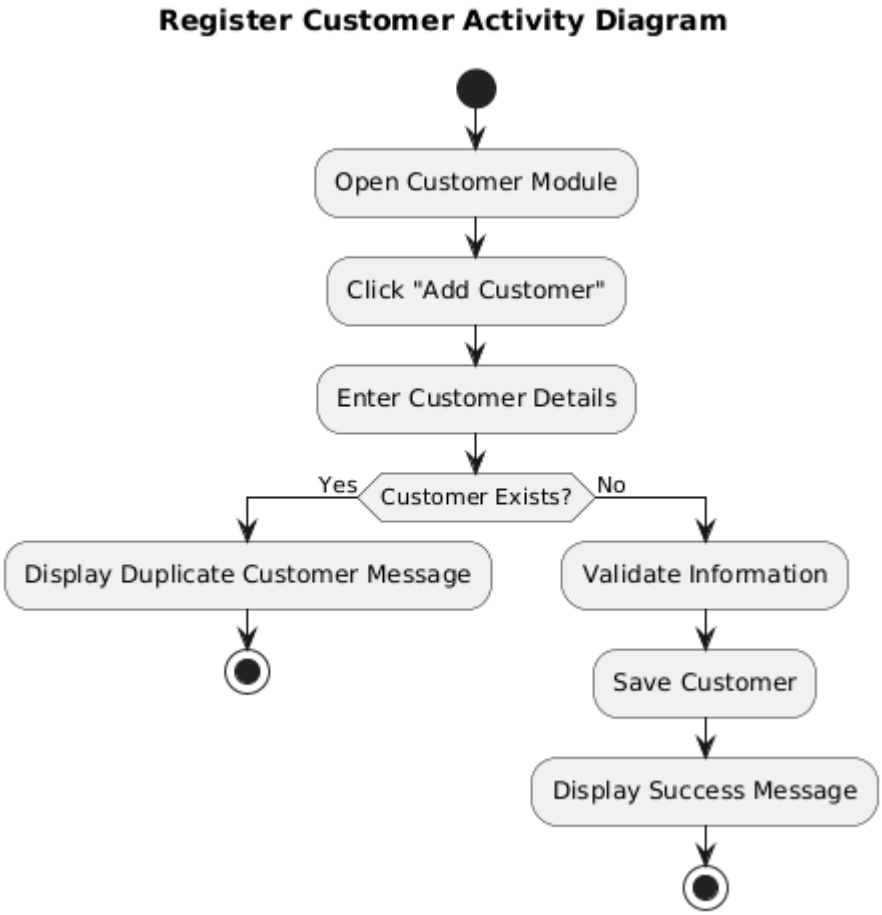
If the hashes match, the user’s operational status is verified. If the account is active, the system generates a signed JSON Web Token containing the user’s role and permission metadata, caches the session token within Redis, and returns a 200 OK response to the client. If any step fails, the server responds with a 401 Unauthorized block and increments the failed login counter.

Figure 5.2.1 – Login Activity



5.2.2 Register Customer Activity

Figure 5.2.2 – Register Customer Activity



- **Purpose:** To create and save an active consumer record within a business tenant's account.
- **Description:** Captures customer profiles, verifies phone or email uniqueness within the tenant context, and seeds the customer ledger.

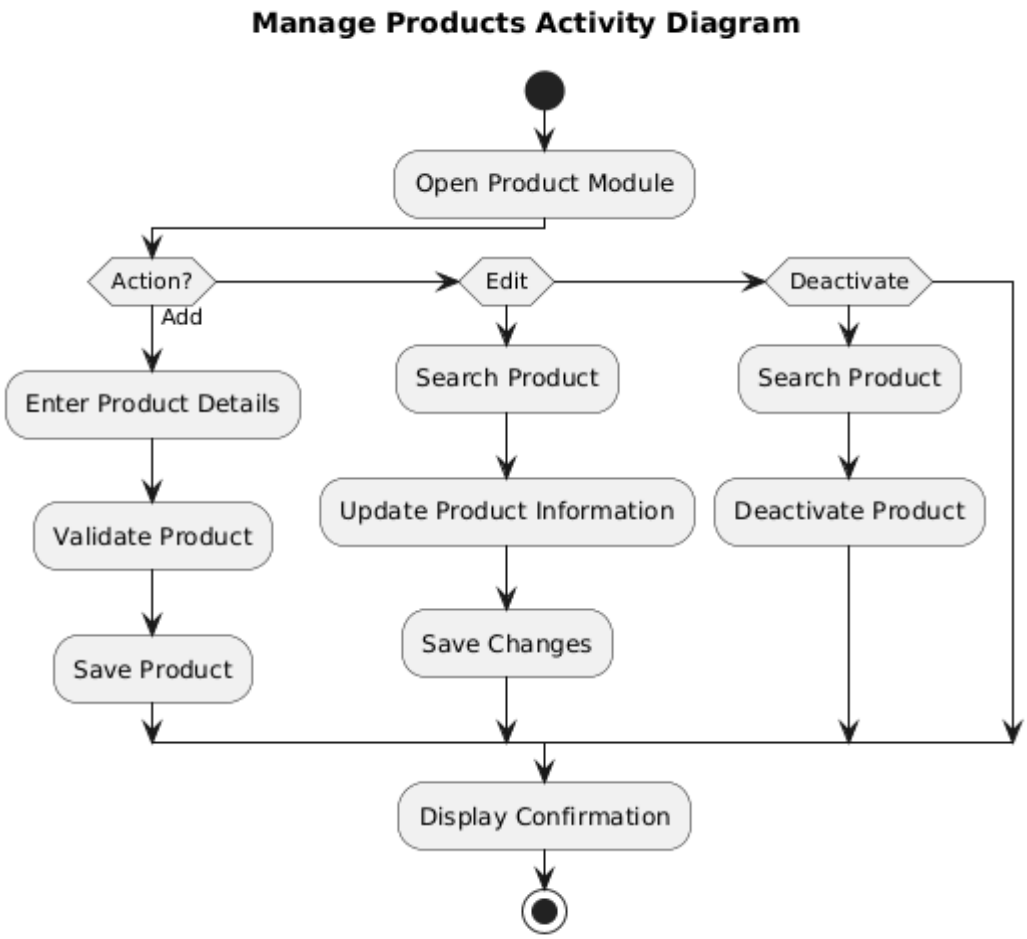
Workflow Explanation:

An authorized user opens the customer management interface and fills out the demographic form (Name, Email, Phone, and Opening Credit Balance). Upon submission, the backend checks for duplicate contacts within that specific business account.

If the email or phone number is already registered to another active customer under the same tenant, the database unique constraint catches it, and the system throws a conflict exception. Otherwise, a new record is saved to the database. The system then returns the freshly generated customer profile object back to the UI view.

5.2.3 Manage Products Activity

Figure 5.2.3 – Manage Products Activity



- **Purpose:** To manage the creation, modification, and structural pricing variables of retail items within the system catalog.
- **Description:** Governs product life cycles, links catalog items to categories, and initializes tracking parameters.

Workflow Explanation:

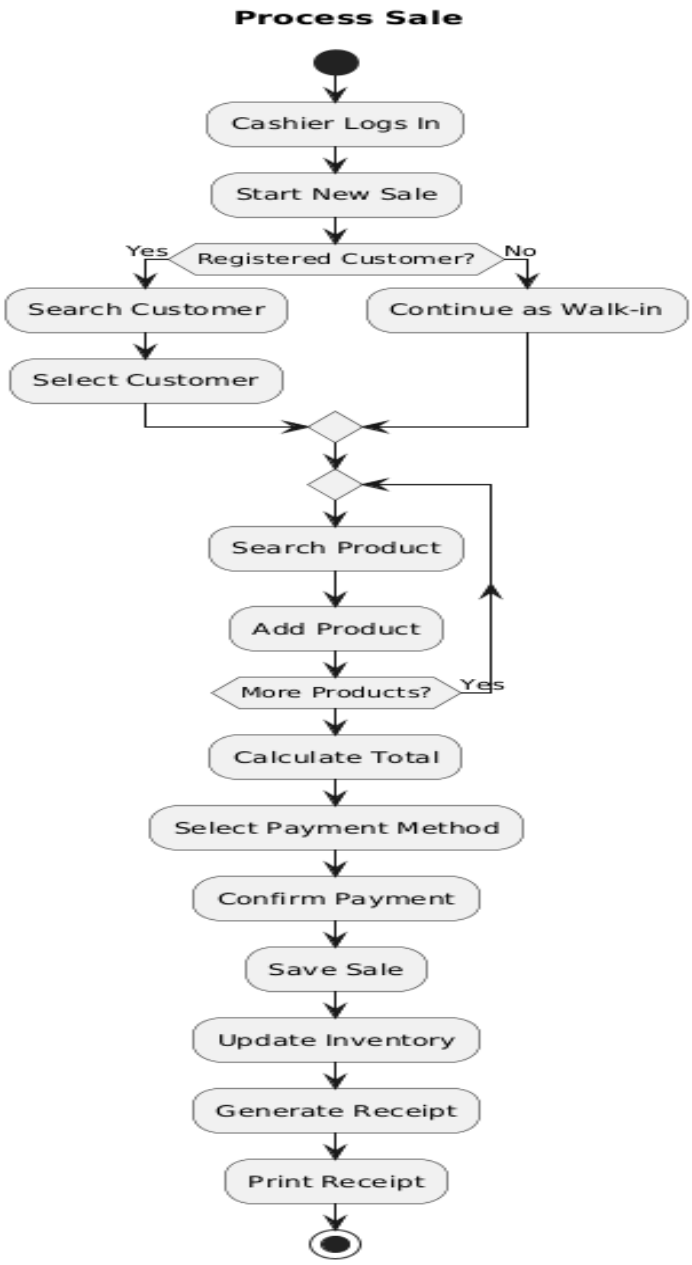
A Manager or Admin submits a product profile change containing the product name, stock keeping unit (SKU), barcode index, cost base, retail price, and category mapping. The NestJS

ProductController routes the request through a dedicated permission guard to confirm the user has catalog write access.

The application validates the data structure and verifies that the provided SKU is unique within that specific business tenant. If the data is valid, the transaction writes the changes to the database catalog. If inventory tracking is enabled for the item, the system automatically creates a matching entry in the inventory_items table with a stock balance of zero.

5.2.4 Process Sale Activity

Figure 5.2.4 – Process Sale Activity



- **Purpose:** To orchestrate checkout processing, manage concurrent item selections, execute payments, and handle real-time inventory deductions.
- **Description:** The core high-concurrency workflow of the system, managing point-of-sale operations, calculating totals, and updating transaction histories.

Workflow Explanation:

A Cashier initiates a checkout by scanning barcodes or selecting items via the UI. The Next.js frontend builds a client-side cart state, computing line-item totals, tax fields, and discounts in real time. Once the cart is confirmed, the cashier submits the transaction payload to the backend /sales controller.

The backend runs a distributed transaction lock across the requested database records to prevent stock allocations from changing during processing. The system verifies that each requested item is in stock. If the check passes, the transaction computes final invoice numbers, saves line items to the database, deducts quantities from the inventory tables, records payment parameters, and returns a success response. If a race condition or stock shortage occurs, the transaction aborts and returns an error message.

5.2.5 Create Purchase Order Activity

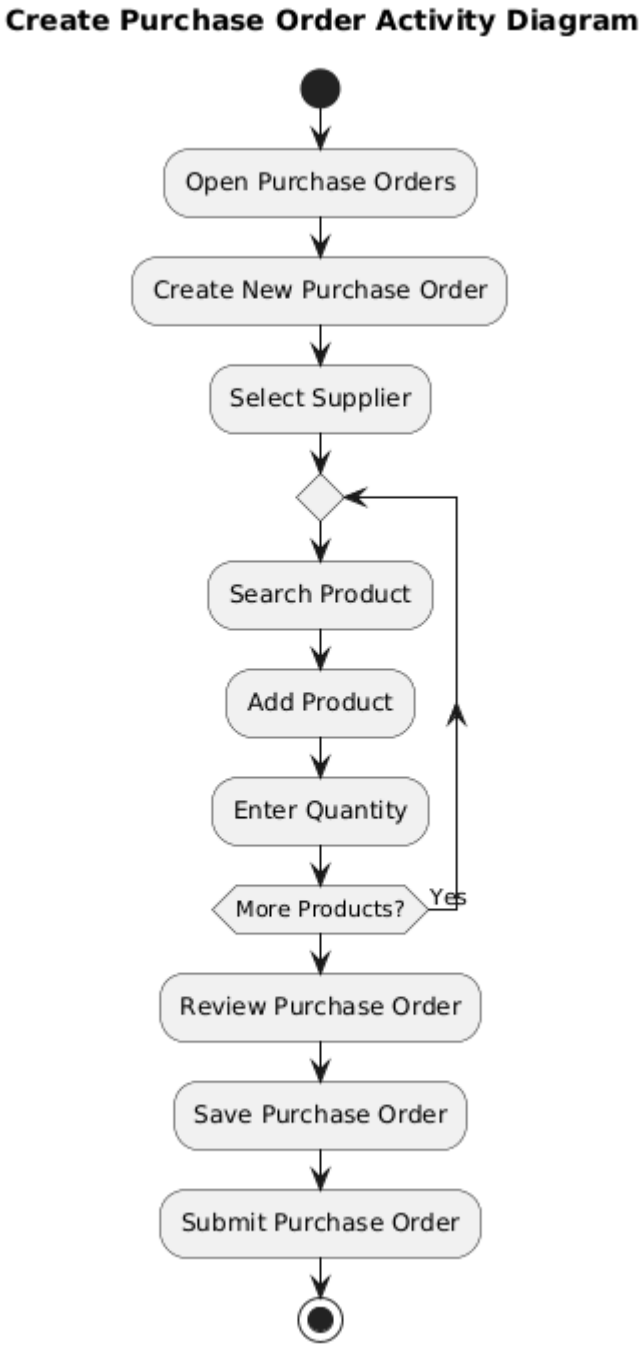
- **Purpose:** To format, approve, and issue structured stock replenishment requests to third-party distribution suppliers.
- **Description:** Manages supply chain procurement pipelines by locking down product items, cost thresholds, and delivery milestones.

Workflow Explanation:

The operator opens the procurement module, selects an active supplier, and appends the requested product variants alongside their agreed unit costs and ordering volumes. The system verifies that the supplier is active and that the unit costs are formatted correctly.

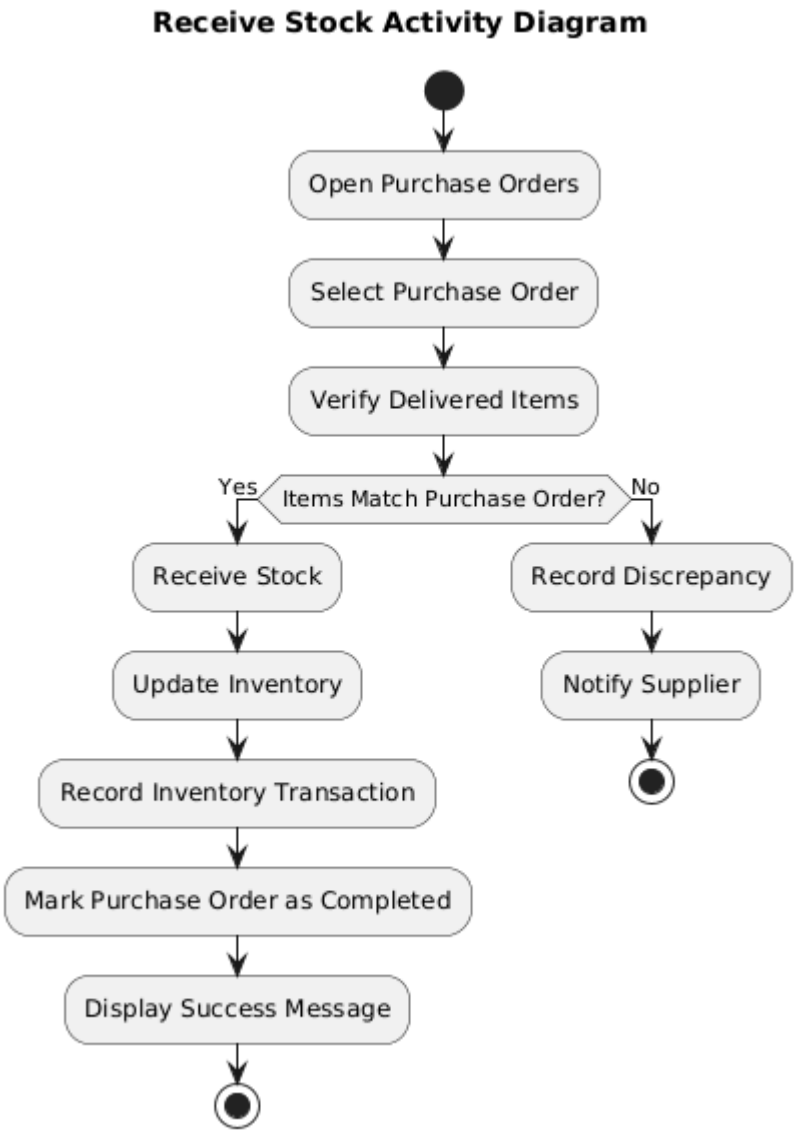
The system saves the order header with an initial DRAFT status. The manager then reviews the order details and clicks approve. This switches the status to SENT, updates the supplier log history, and generates a structured order document ready for distribution.

Figure 5.2.5 – Create Purchase Order Activity



5.2.6 Receive Stock Activity

Figure 5.2.6 – Receive Stock Activity



- **Purpose:** To reconcile incoming physical freight deliveries against open purchase orders, update inventory counts, and log supplier financial liabilities.
- **Description:** Reconciles physical incoming inventory counts against open purchase orders, updates stock counts, and logs supplier financial metrics.

Workflow Explanation:

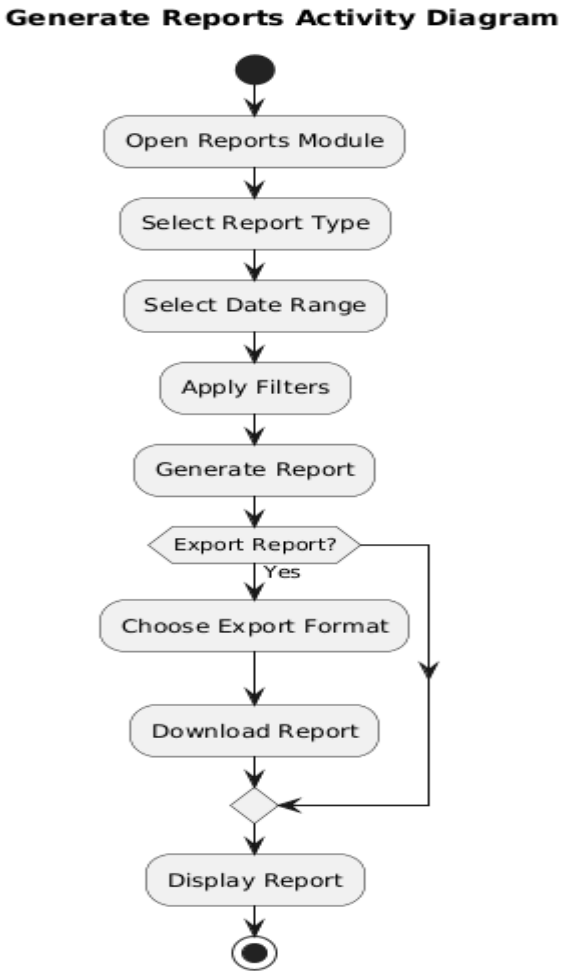
When stock arrives at a facility, an inventory clerk loads the open purchase order records. The clerk inspects the physical boxes and inputs the actual received counts for each line item. The backend

processes the receipt inside an isolated database transaction, comparing the received quantities against the original order.

If any line items show discrepancies, the system flags the order for partial receipt tracking. The database then increments the quantities inside the inventory_items table for the matching products and adjusts the supplier's balance ledger to reflect the new financial liability. Once all items are processed, the purchase order status is updated to RECEIVED.

5.2.7 Generate Reports Activity

Figure 5.2.7 – Generate Reports Activity



- **Purpose:** To aggregate raw historical transaction records into structured business intelligence reports.
- **Description:** Collects data across specific timeframes to generate financial summaries, profit calculations, and stock alert summaries.

Workflow Explanation:

A user requests a report by specifying a date range and type (e.g., Sales Performance, Tax Liabilities, or Low-Stock Alerts). The backend catches the request and applies performance

optimization filters. Instead of scanning active transactional tables line-by-line, the application routes the request to dedicated read-views in the database.

The database aggregates the metrics, calculates totals, and groups the records by business unit. The system then sends the compiled dataset back to the UI, where the Next.js presentation layer renders it into analytical charts and downloadable document summaries.

5.3 Sequence Diagrams

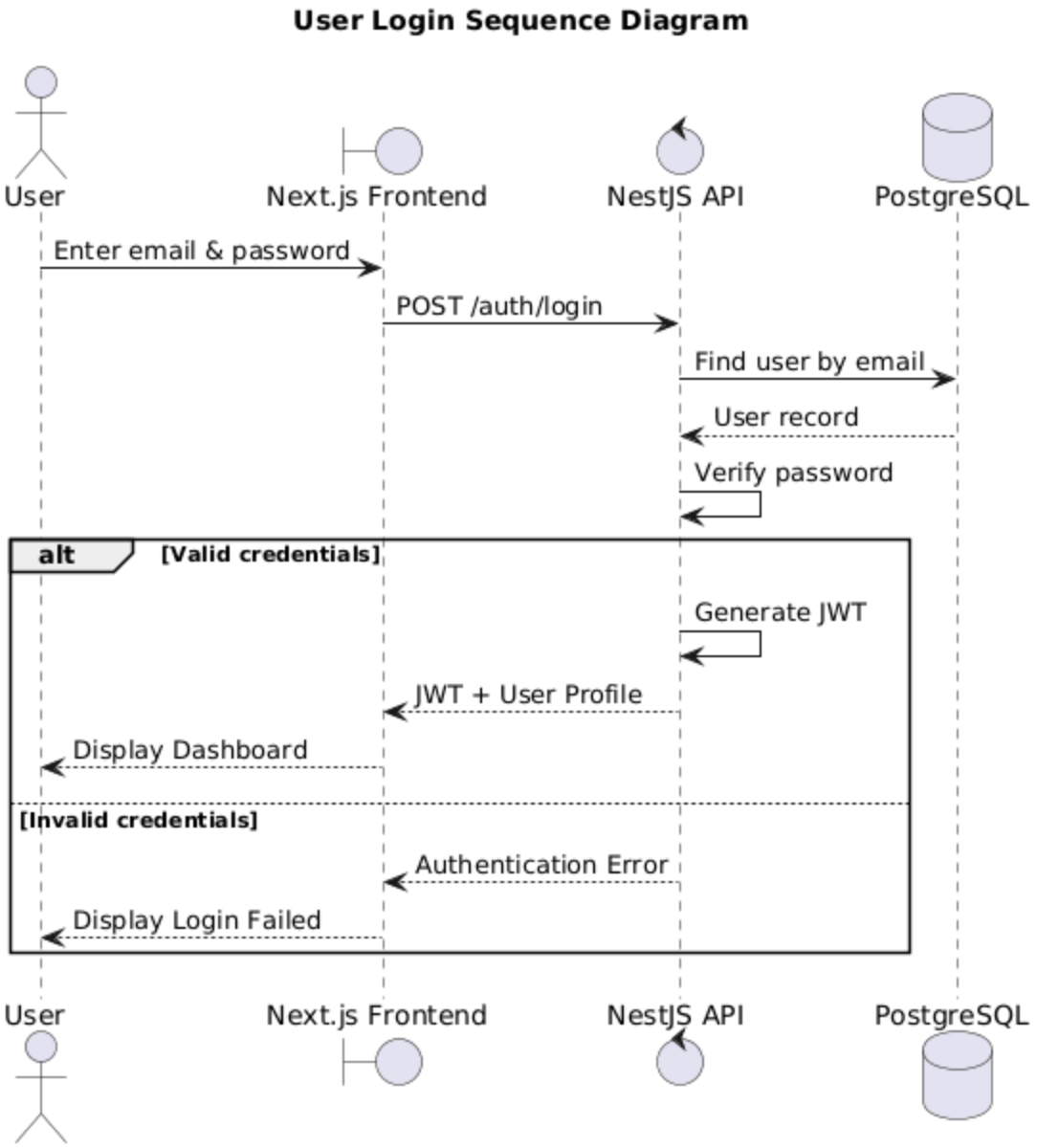
5.3.1 User Authentication (Login Flow)

This workflow details the sequential step-by-step lifecycles of validation, token signing, and session caching across components during user authentication.

The front-end client passes credentials to the AuthController. The controller delegates verification to the AuthService, which calls the UserService to retrieve the corresponding database record. If the account is locked or missing, an error bubbles back to the controller. If the user record is found, the system verifies the password using a cryptographically secure hashing pipeline.

Upon successful verification, the AuthService calls the token factory to sign a new JSON Web Token (JWT). The system registers the active session token within Redis with an explicit time-to-live (TTL) expiration boundary, and then passes the completed token structure back to the user view.

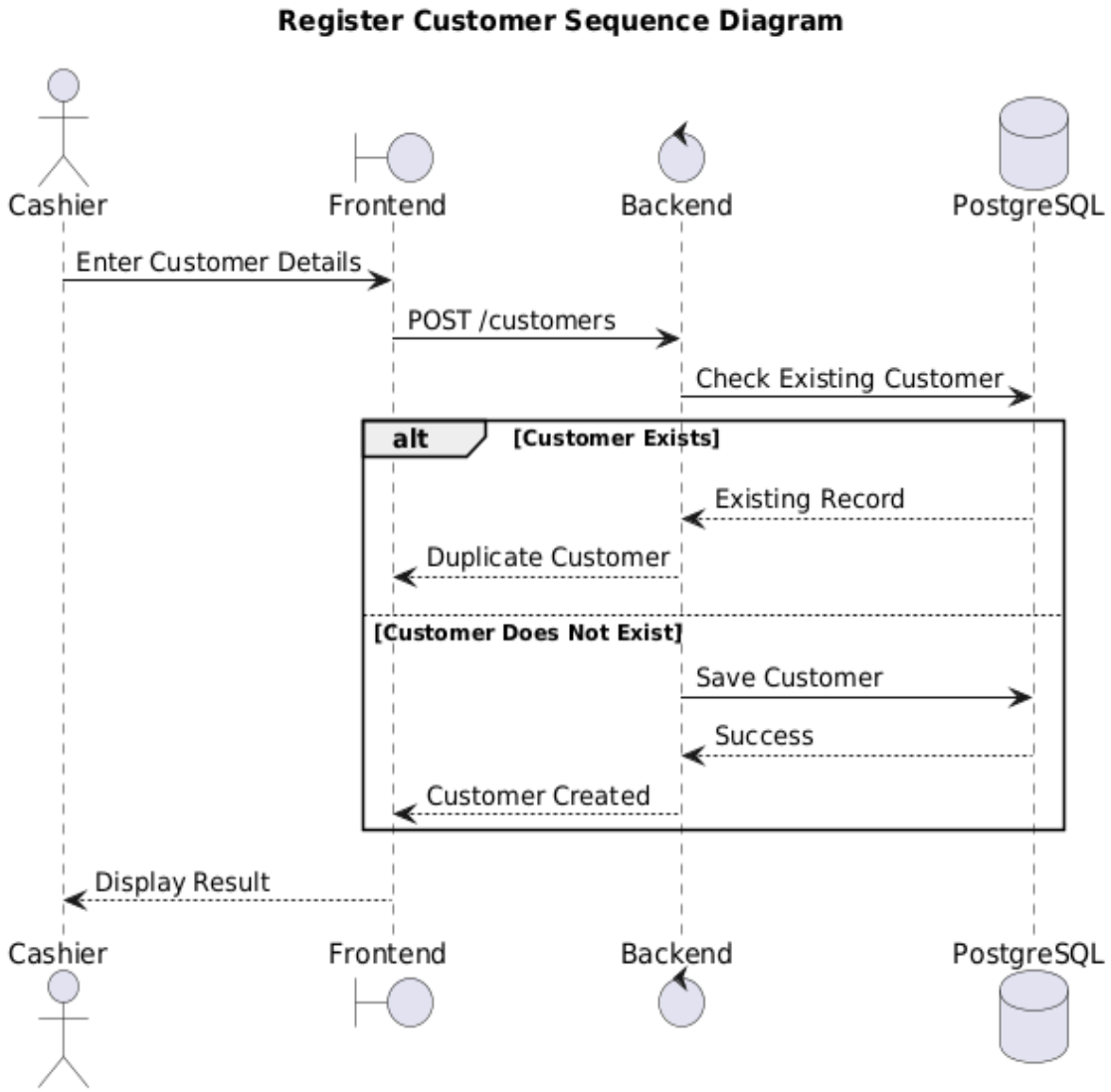
Figure 5.3.1 – User Authentication (Login Flow) Sequence Diagram



5.3.2 Register Customer Flow

Details the sequential execution lifecycle when creating an isolated customer record.

Figure 5.3.2 – Register Customer Flow Sequence Diagram



The frontend client passes customer demographic variables to the backend CustomerController. The controller evaluates the user's active session token using an RBAC interceptor to confirm they have permission to write to the customer ledger. The controller then forwards the payload to the CustomerService.

The service checks the database to verify that the phone and email coordinates are unique within that business tenant's account. If the checks pass, the database saves the new customer record. The application then returns the saved customer object back to the client interface.

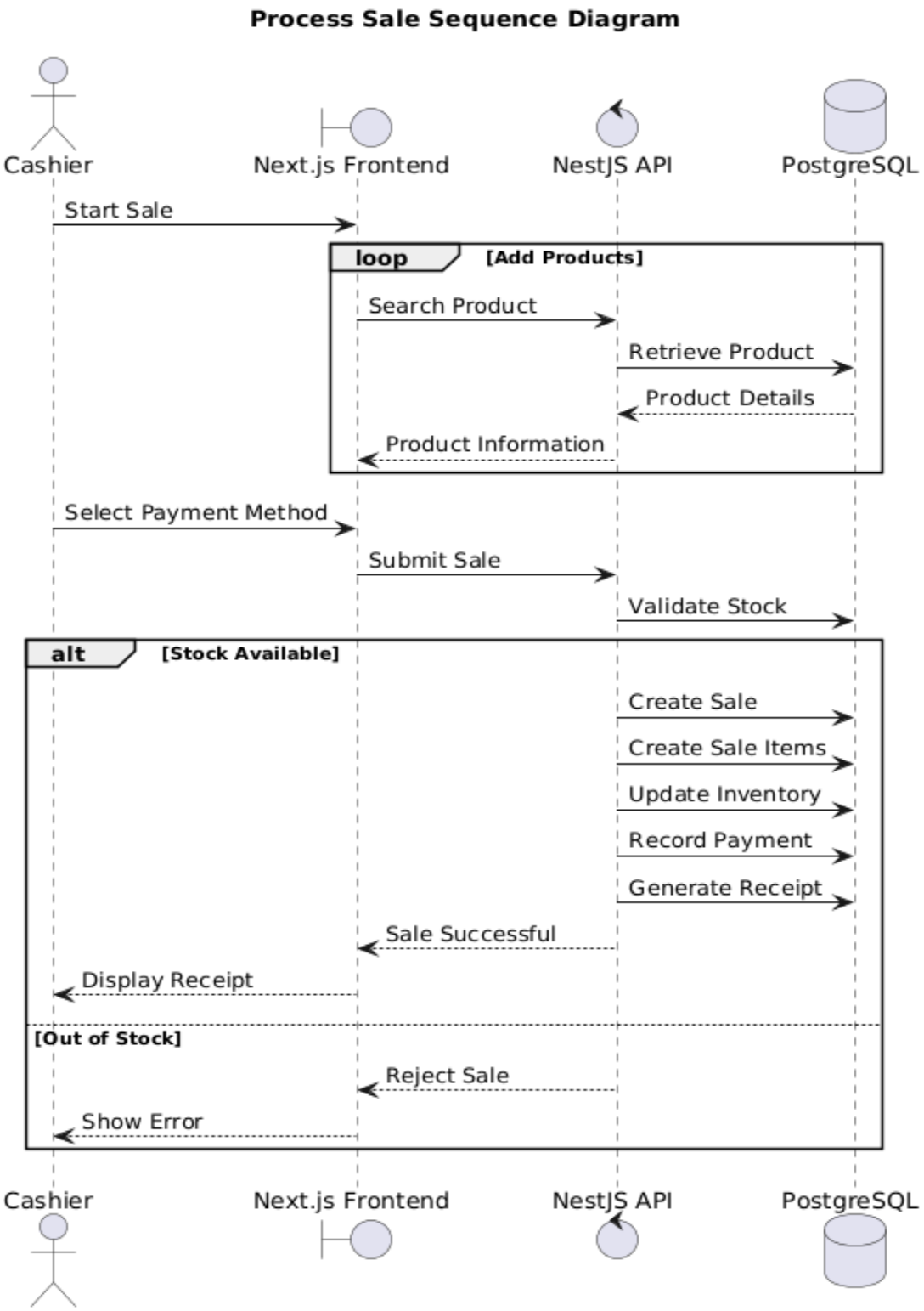
5.3.3 Process Sale Flow

Traces the high-concurrency interaction steps required to execute a point-of-sale checkout transaction.

The checkout interface sends a cart payload to the backend SalesController. The controller runs identity checks and delegates the cart to the SalesService. The service initializes an isolated database transaction block and requests a database lock on the rows inside the inventory_items table for all matching products.

The system verifies available quantities against the cart request. If stock is sufficient, the database creates a new record in the sales table, inserts individual line items into sale_items, updates stock counts, logs payment processing data through the PaymentService, and commits the transaction. The system then returns a success response to the client.

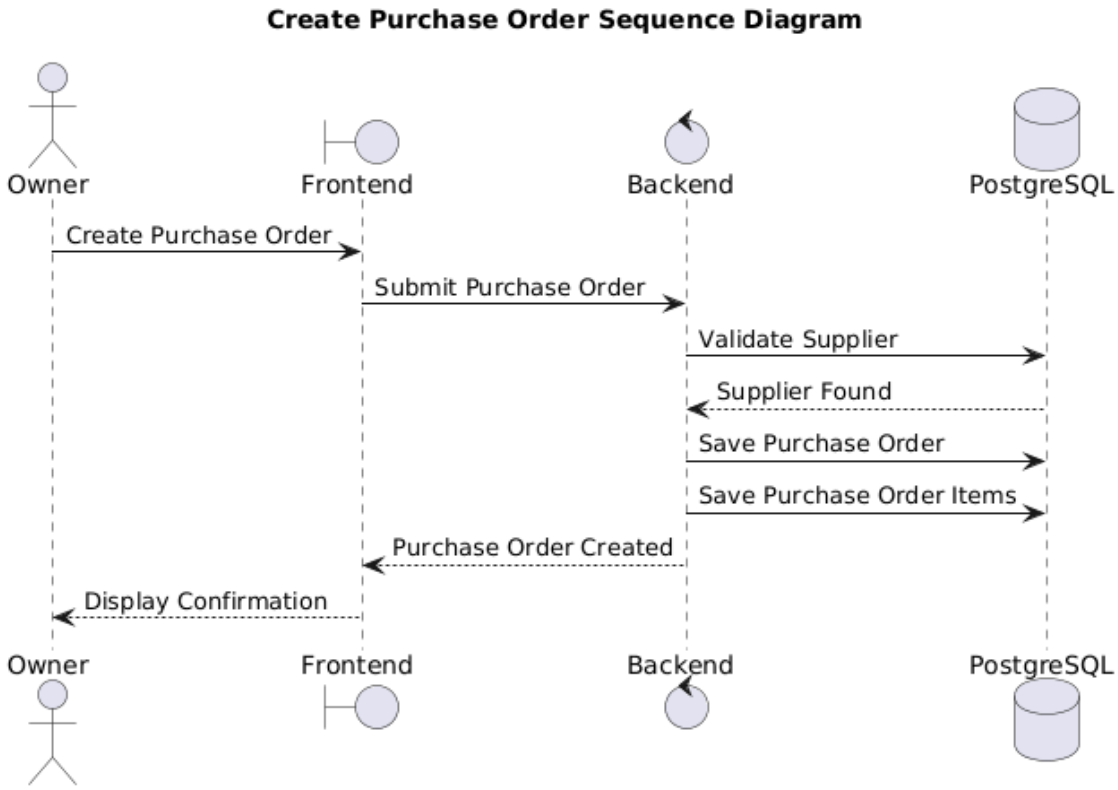
Figure 5.3.3 – Process Sale Flow Sequence Diagram



5.3.4 Create Purchase Order Flow

Traces procurement interactions between management roles and supply chain systems.

Figure 5.3.4 – Create Purchase Order Flow Sequence Diagram



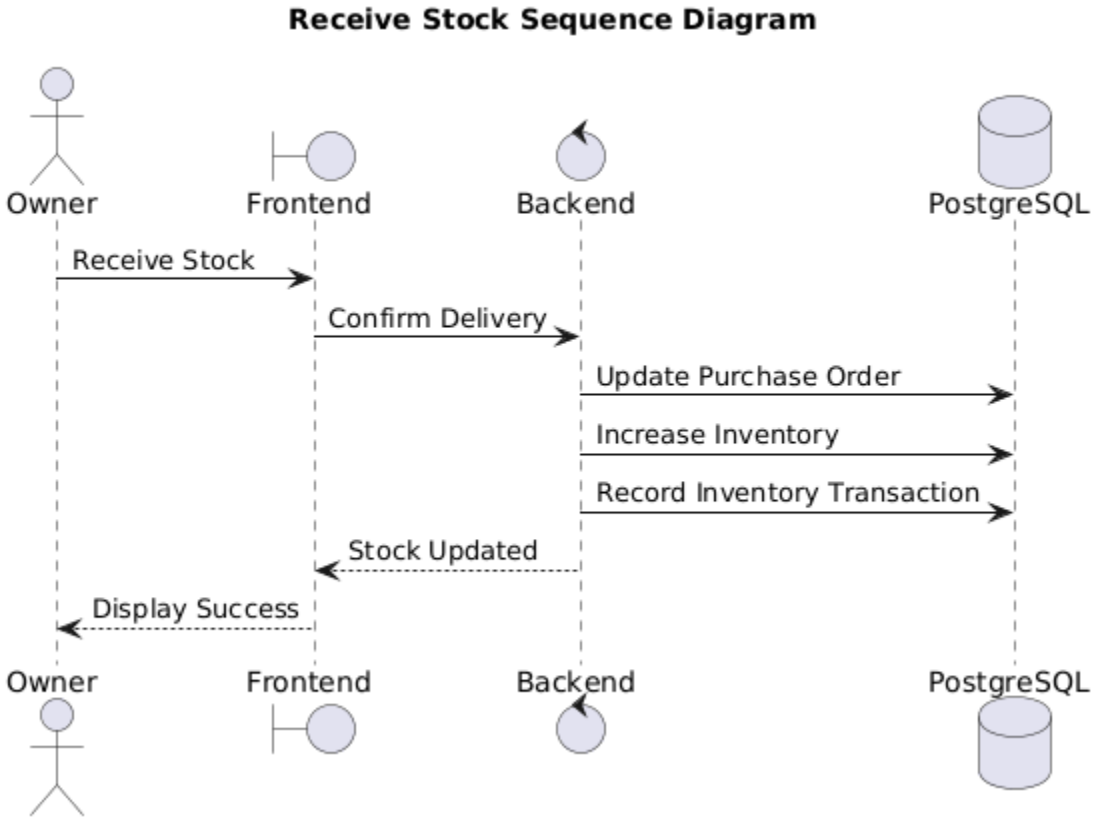
The user designs a stock requisition order inside the procurement interface, which transmits the details to the PurchaseOrderController. The controller passes the request to the PurchaseOrderService, which checks the supplier directory to ensure the selected partner is active. The service creates a new entry in the purchase_orders table with a status of DRAFT and saves the requested line items to the database.

When a manager issues an approval command, the service verifies their access permissions, changes the status to SENT, locks the unit cost points, and generates an immutable purchase order structure.

5.3.5 Receive Stock Flow

Traces structural updates when incoming inventory deliveries arrive at a facility.

Figure 5.3.5 – Receive Stock Flow Sequence Diagram



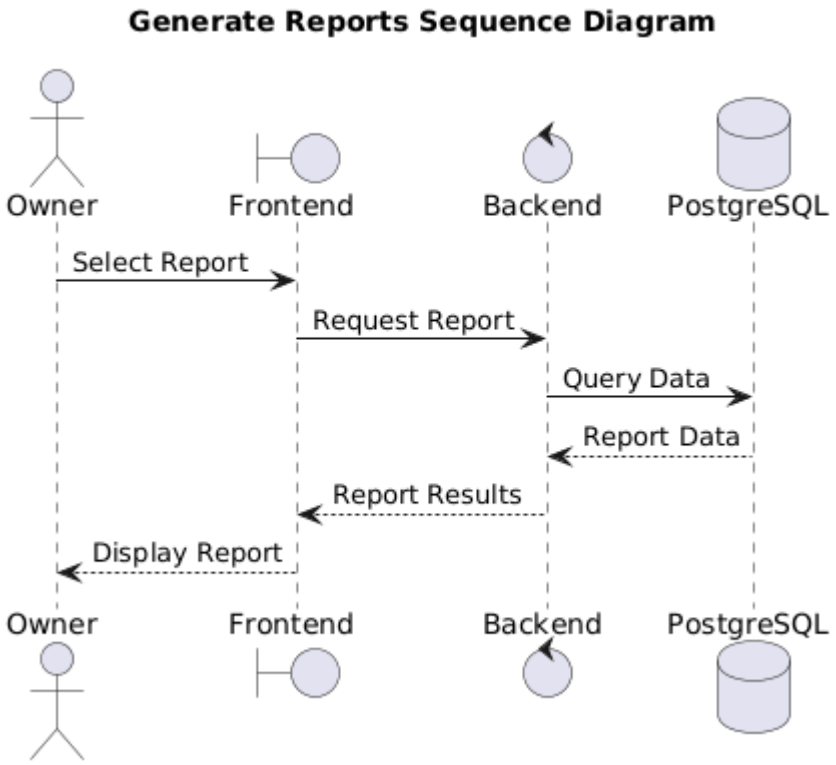
The operator selects an open purchase order and enters the physical delivery counts into the incoming inventory form. This form submits the data to the InventoryController, which routes it to the InventoryService. The service starts a secure database transaction, loads the purchase order, and verifies the incoming counts.

The database increases the item counts inside the inventory_items table, logs the adjustments in the inventory history tracking tables, calculates the updated total valuation of the warehouse assets, adjusts the supplier's balance ledger, and sets the purchase order status to RECEIVED.

5.3.6 Generate Reports Flow

Traces the steps for aggregating data to build business intelligence analytics.

Figure 5.3.6 – Generate Reports Flow Sequence Diagram



The dashboard requests an analytical aggregation by passing parameters to the AnalyticsController.

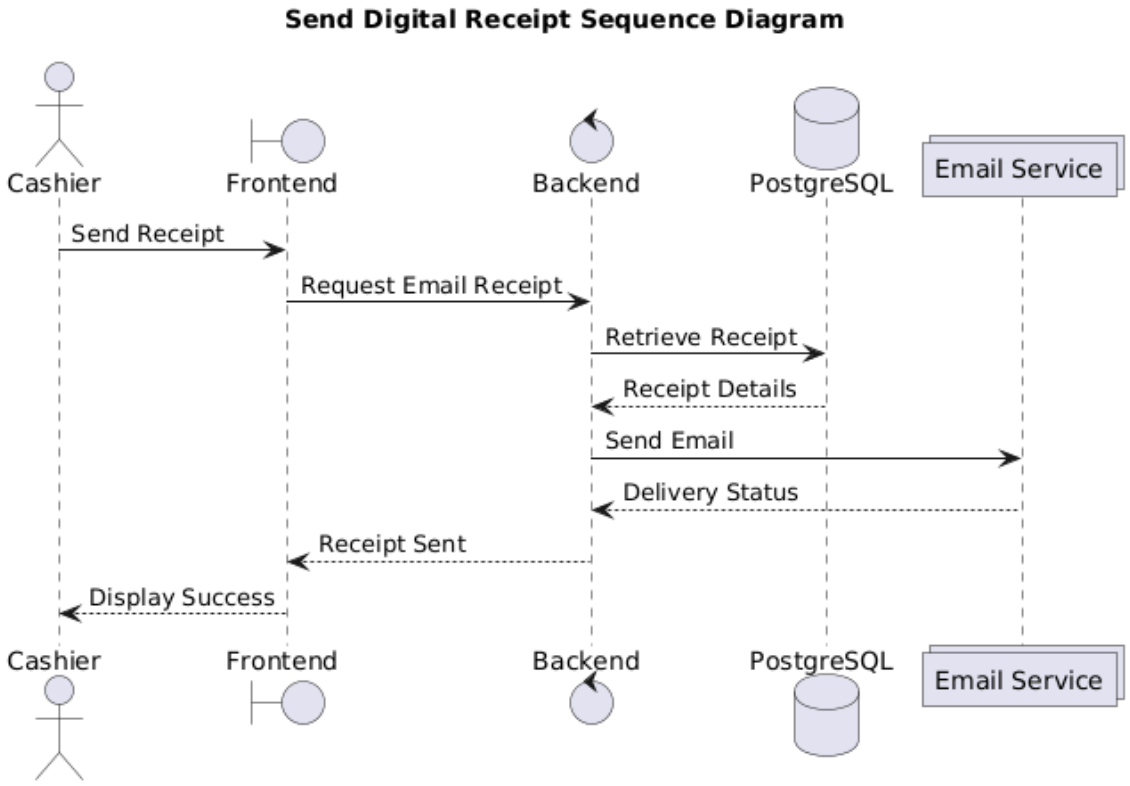
The controller checks the user's role permissions and passes the parameters to the AnalyticsService. The service runs optimized queries against the database's read-only reporting views to bypass transactional database bottlenecks.

The database runs the aggregations, calculates financial totals, and groups the datasets. The service packages these metrics into a clean structure and passes them back to the client interface to render dashboard charts.

5.3.7 Send Digital Receipt Flow

Traces the asynchronous pipeline for generating and sending digital invoices.

Figure 5.3.7 – Send Digital Receipt Flow Sequence Diagram



When a sales transaction is successfully committed, the SalesService emits a background event message containing the sale ID to the system's event bus. The ReceiptNotificationConsumer picks up this event asynchronously, shielding the core POS transaction loop from notification processing delays.

The consumer passes the ID to the ReceiptGenerationService, which pulls the transaction header, line item lists, and corporate profile parameters from the database. The service formats the data into a clean structure, calls external communication gateways to deliver the receipt via email or SMS, logs the delivery status, and completes the background task.

5.4 State Diagrams

5.4.1 Purchase Order Lifecycle State Machine

A purchase order tracks procurement milestones, moving through a strict series of operational states:

```
[ DRAFT ] --(Manager Approval)--> [ SENT ] --(Physical Delivery)--> [
RECEIVED ]
   |                                  |
   +-------------(Cancel)-------------+-------------(Cancel)--> [ CANCELLED
]
```

DRAFT: The initial creation state. Line items and quantities can be freely edited, modified, or deleted. The order has no impact on supplier balances or active stock levels.

SENT: The order is locked and issued to the distributor. Items and costs can no longer be edited. The system is now actively waiting for physical delivery.

RECEIVED: The terminal fulfillment state. Incoming stock is verified, items are added to inventory records, and financial liabilities are logged in the supplier ledger. The order is locked permanently.

CANCELLED: The order is aborted. This state can be triggered from either DRAFT or SENT, invalidating the document and preventing any future stock receipts against it.

5.4.2 Sale Transaction State Machine

Tracks the operational stages of a consumer sales transaction:

```
[ PENDING ] --(Payment Success)--> [ COMPLETED ]
    |
    +--(Payment Failure / Timeout)--> [ FAILED ]
```

PENDING: The invoice header is created, line items are saved, and the system is waiting for payment validation.

COMPLETED: The payment is cleared. The system permanently saves the sale record, updates inventory levels, and releases all database row locks.

FAILED: The checkout is aborted due to payment failures, timeouts, or network drops, releasing any reserved stock back to active inventory.

5.4.3 Payment Settlement State Machine

Tracks the clearance status of financial payments linked to transactions:

```
[ UNPAID ] --(Initiate Gateway Authorization)--> [ PROCESSING ]
                                                      |
                                +---------------------+--------------------
----+
                     (Callback Success)                                (Callback
Denied)
                                v
v
                        [ SETTLED ]                                        [
DECLINED ]
```

UNPAID: The initial state indicating a transaction balance is outstanding.

PROCESSING: The payment details are submitted to an external clearing gateway, and the system is waiting for an asynchronous authorization callback.

SETTLED: The financial amount is successfully cleared, updating internal cash ledgers and marking the payment as fully resolved.

DECLINED: The clearing house rejects the payment due to insufficient funds or authentication failures, requiring the cashier to select an alternative payment method.

5.4.4 Product Availability State Machine

Tracks the status of a catalog item within the enterprise registry:

```
[ ACTIVE ] --(Deactivate Command)--> [ INACTIVE ] --(Discontinue Command)-->
[ ARCHIVED ]
```

ACTIVE: The product is available for business operations. It can be looked up by scanning barcodes, added to active point-of-sale carts, and appended to new purchase orders.

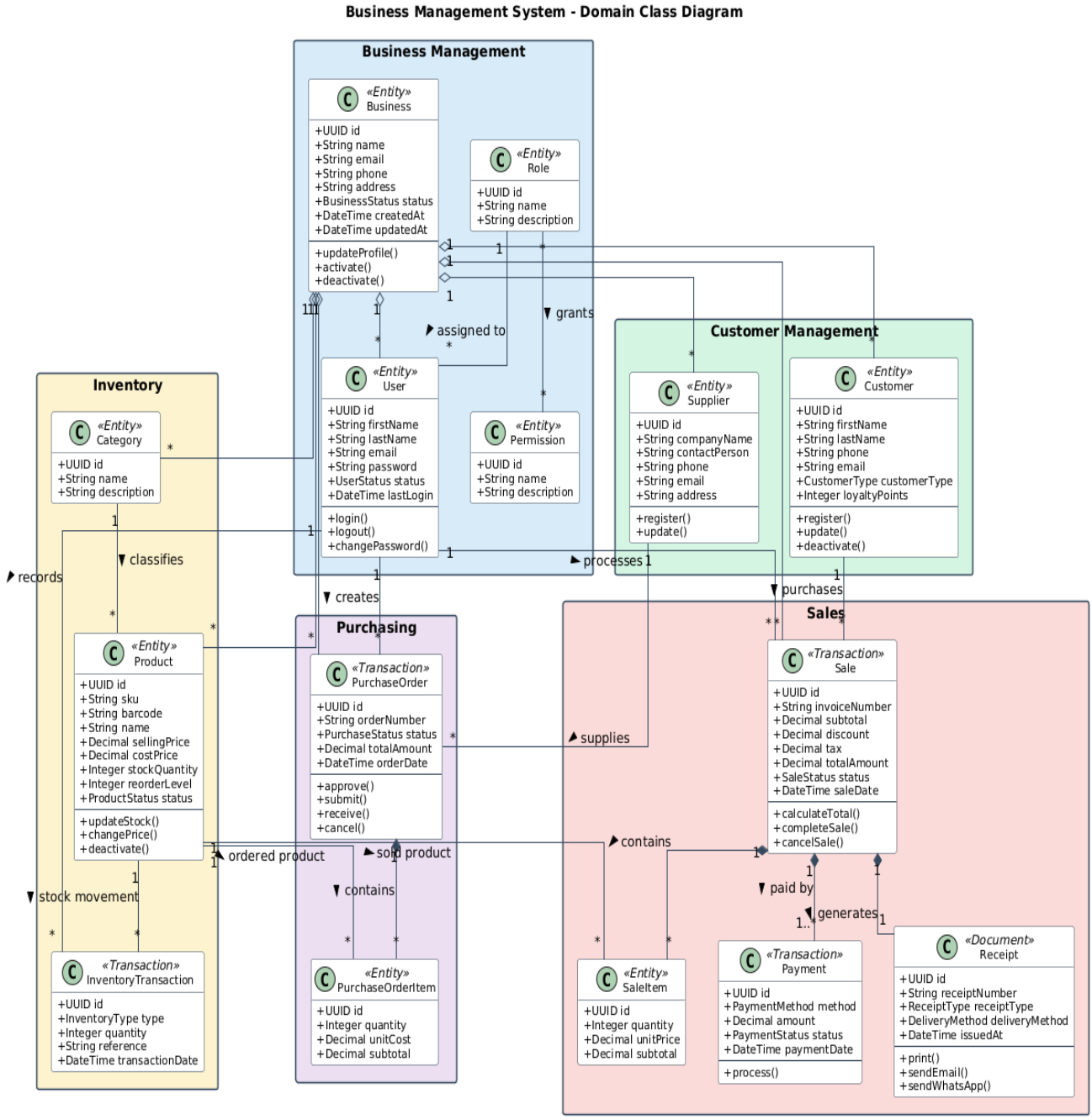
INACTIVE: The product is temporarily hidden from active checkout operations, but existing records remain intact for inventory auditing.

ARCHIVED: The product is permanently retired from operations. It is hidden from all configuration views and point-of-sale systems, but retained in the database to preserve historical reporting and transactional continuity.

6. STRUCTURAL DESIGN

This chapter describes the static code structures and object models that implement the logical domain semantics of the Business Management System (BMS). The object model is designed using NestJS TypeScript classes to bridge the presentation view state with the underlying database tables via Prisma ORM contracts.

Figure 6.1 – Class Diagram



6.1 Domain Entities and Model Specifications

6.1.1 Core Classes

The object structure translates normalized persistence tables into domains containing attributes, access modifiers, and operational data transformations:

- **Business:** The root configuration entity. It manages high-level company properties such as business identification mappings, tax compliance rates, active currencies, and branch localized values.
- **User:** The workforce identity entity. It maintains information for active system operators, securely locking down email credentials, encrypted password hashes, status indicators, and role designations (ADMIN, MANAGER, CASHIER).
- **Product:** The core catalog entity. It contains details for specific warehouse variants, including names, Stock Keeping Units (SKUs), structural barcodes, buy cost profiles, and retail prices.
- **InventoryItem:** The physical asset entry. It isolates stock parameters from catalog listings, maintaining real-time quantity attributes and minimum restock thresholds.
- **Sale:** The primary transaction header class. It tracks point-of-sale operational sessions and saves customer allocations, cumulative balances, total tax amounts, and invoice sequences.
- **SaleItem:** The individual item detail class. It acts as an active line item that maps a snapshot of product prices and quantities against a specific transaction.

6.2 Relational Coupling and Structural Mappings

The entities are connected using explicit association models to ensure data integrity and prevent cross-tenant data leaks:

- **One-to-Many Associations:** The Business entity serves as the root parent, maintaining independent array lists for User[], Product[], Customer[], Supplier[], and Sale[] instances. This structural configuration isolates multi-tenant operational data at the class instantiation layer.
- **Composition and Aggregation:** The Sale instance uses strong structural composition to manage its list of SaleItem[] rows. A SaleItem cannot exist independently from its parent Sale instance; deleting a checkout header automatically removes all dependent line items from memory.
- **Reference Associations:** The SaleItem class maintains a reference association to the Product entity. This link provides immediate access to item information during the cart compilation loop without introducing tight class inheritance structures.

6.3 Domain Responsibilities and Operational Design Patterns

The system’s structural layout uses dedicated object-oriented design patterns to handle complex business logic cleanly:

- **Encapsulated State Validation:** Domain classes handle their own state safety checks. For example, the Product entity contains methods that prevent its retail price from falling below

its cost base, catching structural invalidations before they reach database persistence transactions.

- **Decoupled Domain Data Structures:** Objects are split into thin persistence entities and active data transfer layers. This approach isolates external network operations from the system's core object properties, preventing presentation view adjustments from altering backend database models without passing through proper business services.

7. INTERFACE DESIGN

This chapter describes the interface architecture engineered to facilitate secure, decoupled, and low-latency communication between the Next.js client tier and the NestJS application service engine.

7.1 REST API Architecture

The external communication interface of the Business Management System (BMS) is designed as a stateless RESTful Web API. The system enforces strict architectural compliance with standard HTTP methods to represent explicit CRUD (Create, Read, Update, Delete) state transitions over domain resource paths:

- **GET:** Retrieves a representation of a resource or collection without modifying downstream database state.
- **POST:** Formulates a new resource instance within the persistent relational engine.
- **PUT:** Replaces the entirety of an existing resource payload with updated data structures.
- **PATCH:** Appends partial delta modifications to a localized resource entity without resetting unmentioned properties.
- **DELETE:** Terminates or deactivates an existing resource allocation within the database.

Resource identifiers use plural nouns structured uniformly in lowercase syntax (e.g., /api/v1/products, /api/v1/sales).

7.2 Authentication Flow

API interaction requires authentication using an asynchronous, cryptographically signed JSON Web Token (JWT) workflow. Upon successful verification of workforce credentials via the /api/v1/auth/login endpoint, the backend engine returns a dual-token payload consisting of a short-lived Access Token and a long-lived Refresh Token. The client application must append the Access Token to the header of every outbound HTTP request using the standard Bearer scheme:

```
Authorization: Bearer <JWT_ACCESS_TOKEN>
```

The NestJS server validates the token on each incoming request using an entry-intercepting JwtAuthGuard. The token signatures are decoded using shared asynchronous cryptographic keys to unpack embedded authorization variables, such as user identity properties, tenant corporate scoping rules, and RBAC permission structures.

7.3 Data Transfer Validation Mechanics

To protect the core business services from raw, structurally broken payload entries, all inbound network payloads must pass through a strict validation pipeline. The system uses the NestJS global ValidationPipe combined with declarative class-validator annotations on Data Transfer Objects (DTOs). When a client submits a payload, the validation pipeline automatically checks the data type, character length, numeric range, and structural nullability before invoking any domain

controllers. If the payload violates these constraints, the pipeline intercepts the request, aborts execution, and immediately returns an HTTP status code 400 Bad Request.

7.4 Unified Error Exception Handling

The system uses an explicit, centralized approach to manage system exceptions, preventing raw runtime application crashes or sensitive database engine errors from leaking across network boundaries. The application intercepts failures using a global NestJS HttpExceptionFilter. When an exception occurs during execution, the filter catches the error and formats it into a uniform JSON response structure. This ensures that regardless of whether an error is thrown by database constraints, authentication failures, or business logic rule violations, the client application receives a standardized error payload:

```
{
  "statusCode": 409,
  "timestamp": "2026-07-08T12:39:00.000Z",
  "path": "/api/v1/products",
  "message": "Product SKU already exists within this business tenant.",
  "error": "Conflict"
}
```

7.5 Standardized API Response Layout

To minimize the parsing logic required on the frontend presentation layer, all non-error API responses match a consistent envelope structure. Successful transactions return payload envelopes that separate primary entity datasets from metadata blocks:

```
{
  "success": true,
  "data": {
    "id": "e3088aaa-1533-4131-b033-253b1a5c8728",
    "sku": "PROD-WHISKEY-01",
    "name": "Premium Barrel Aged Whiskey",
    "price": 85.00
  },
  "meta": {
    "processingTimeMs": 14
  }
}
```

For paginated read queries, the meta envelope expands to include explicit pagination details, such as totalItems, itemCount, perPage, totalPages, and currentPage.

7.6 API Versioning Policy

To support continuous deployment without risking breaking updates for active front-end instances, the system uses URL-based API versioning. The base routing prefix includes an explicit version identifier (/api/v1/). If a future feature requires backward-incompatible modifications to data models or endpoint contracts, engineers can deploy a parallel path (/api/v2/) alongside the existing endpoint. This allows legacy and updated client structures to co-exist safely during transition periods.

7.7 Automated OpenAPI Specification (Swagger)

The complete surface layout of the REST API is documented automatically using the OpenAPI 3.0 specification standard. NestJS decorators (e.g., @ApiTags(), @ApiOperation(), @ApiResponse()) are applied directly across controllers and DTO files. During the application compilation step, the framework generates a complete JSON schema definition file. This schema is exposed via an interactive web interface at the /api/docs route, providing developers with a live sandbox environment to view, test, and integrate with every available backend route.

8. SECURITY DESIGN

This chapter describes the comprehensive defense architecture designed to secure the application runtime, client interfaces, network boundaries, and multi-tenant data states of the Business Management System (BMS).

8.1 JWT Authentication and Session Security

User authentication relies on a stateless, cryptographically signed JSON Web Token (JWT) architecture. The authentication engine issues two separate tokens upon successful authorization:

- **Access Token:** Short-lived token signed with an explicit 15-minute expiration boundary, containing user profile properties, multi-tenant scopes, and role allocations.
- **Refresh Token:** Long-lived token signed with a 7-day expiration boundary, used exclusively to acquire a new access token without requiring re-entry of user credentials.

Tokens are signed using the Asymmetric RSA-256 (RS256) cryptographic algorithm. The private key remains securely inside the backend application cluster to sign payloads, while the public key is distributed safely across peripheral nodes to decrypt and verify signatures. To prevent replay attacks, refresh tokens are hashed and tracked in an active Redis blacklist cache upon logout or session revocation.

8.2 Password Hashing Guidelines

Plaintext user credentials are never stored within the persistent relational database engine or exposed inside transient debugging log trails. The system processes password payloads through a cryptographically secure Argon2id or bcrypt hashing pipeline using an optimal work factor cost (e.g., a minimum parameter of 12 rounds for bcrypt). The hashing algorithm automatically appends a cryptographically secure random value (salt) to each plaintext password string prior to verification processing. This prevents pre-computed dictionary matches and rainbow table exploits. The system also limits the size of password strings at the API controller boundary to block buffer overload attempts.

8.3 Role-Based Access Control (RBAC)

The system uses a declarative Role-Based Access Control (RBAC) model to enforce authorization boundaries. Permissions are assigned to specific system roles rather than directly to individual users:

- **CASHIER:** Authorized exclusively to interact with the POS sales and customer registry interfaces.
- **MANAGER:** Extends cashier capabilities with access to product pricing matrices, supplier records, and inventory tracking interfaces.
- **ADMIN:** Possesses global operational control, including the ability to provision accounts, update tenant variables, alter billing profiles, and override system tables.

The NestJS backend enforces these boundaries using custom decorators (e.g., @Roles(Role.MANAGER)) combined with an execution interceptor (RolesGuard). This guard extracts the user's role metadata from the incoming verified JWT payload and checks it against the route's permission matrix before allowing the request to proceed to the domain services.

8.4 Input Validation and Content Sanitization

To defend the application against malformed data and malicious payloads, the system applies strict validation at the network boundary. Incoming JSON structures pass through a processing pipeline using class-validators and class-transformers. String attributes undergo explicit content type verification and character escaping to remove structural HTML tags, script delimiters, or unauthorized character sets. The system uses strict object whitelisting (stripUnknownProperties: true), which drops any extra parameters included in a request payload that are not explicitly defined in the Data Transfer Object (DTO) before the data reaches internal logic components.

8.5 HTTPS Transport Security

The system requires complete Transport Layer Security (TLS v1.3) across all network communication paths. The Nginx edge proxy blocks standard unencrypted HTTP traffic (Port 80) and redirects requests to HTTPS (Port 443). The server enforces cryptographic cipher suites that support Perfect Forward Secrecy (PFS), ensuring that past session data remains secure even if long-term private keys are compromised. Transport security is further reinforced via HTTP Strict Transport Security (HSTS) headers, which instruct modern browsers to interact with the domain exclusively over encrypted channels.

8.6 SQL Injection Prevention

The system is protected against SQL injection attacks by separating query logic from user-supplied data inputs. The application uses Prisma ORM as its primary data access layer, which converts entity parameters into parameterized SQL queries automatically. When complex reporting routines require writing raw database strings, the code uses type-safe SQL template literals (e.g., prisma.$queryRaw using standard tagged templates). This setup passes data inputs as independent parameters to the PostgreSQL engine, preventing the relational core from executing malicious user text as database commands.

8.7 Cross-Site Scripting (XSS) Mitigation

The system applies multiple defense layers to protect users against Cross-Site Scripting (XSS) attacks. The front-end user interface is built on Next.js, which automatically escapes dynamic text content rendered within the Document Object Model (DOM) to neutralize malicious scripts. For workflows that handle rich text or variable customer data strings, the backend validates and sanitizes inputs before saving them to the database. Security is further hardened by appending a strict Content Security Policy (CSP) header through Nginx, which restricts script execution to verified source domains and disables inline script capabilities.

8.8 Cross-Site Request Forgery (CSRF) Defenses

The system uses a stateless REST architecture that passes authentication tokens inside the standard HTTP Authorization bearer header rather than depending on automatic browser cookie transmission. Because web browsers do not automatically append bearer headers to cross-origin requests, this setup inherently neutralizes standard Cross-Site Request Forgery (CSRF) exploit vectors. For auxiliary web tools or static server-side assets that require cookie storage, the system applies strict security flags, including HttpOnly, Secure, and SameSite=Strict.

8.9 Essential Security Headers Configuration

The Nginx edge routing layer injects a robust suite of industry-standard security headers into every outbound HTTP response. These headers harden the application against advanced frontend vulnerabilities:

- **X-Frame-Options: DENY:** Prevents clickjacking attacks by blocking the user interface from being rendered inside external iframes.
- **X-Content-Type-Options: nosniff:** Forces the browser to strictly follow the MIME types defined in the content headers, preventing text files from being executed as scripts.
- **Referrer-Policy: strict-origin-when-cross-origin:** Limits the amount of sensitive referral data exposed to external websites during link navigation.

8.10 Structured Audit Logging

The application uses an immutable audit logging pipeline to record system activity and support forensic compliance tracking. Critical business actions — such as failed authentication attempts, manual inventory adjustments, changes to product pricing, or transaction overrides — are intercepted and logged to an independent storage file. Each audit log entry captures a complete contextual snapshot, including a high-precision timestamp, the target tenant identity, the responsible operator ID, the source IP address, and the precise state delta. The audit database tables restrict modifications, blocking UPDATE or DELETE operations to ensure the integrity of historical logs.

9. DEPLOYMENT DESIGN

This chapter describes the provisioning blueprints, containerization layout, reverse proxy configurations, and high-availability deployment mechanics designed for the Business Management System (BMS).

9.1 Docker Containerization Strategy

The complete software ecosystem is containerized to ensure absolute runtime predictability from local development environments to production-grade cloud clusters. Every component is wrapped into an independent, minimal Linux environment using optimized multi-stage Docker builds. The Next.js presentation frontend and NestJS application backend use Alpine Linux base images to reduce vulnerabilities and minimize container footprints. Dependencies are compiled inside a temporary build container, and only production assets are copied into the final runtime container to minimize storage overhead.

9.2 Orchestrated Container Topologies (Docker Compose)

For baseline deployment and automated stack orchestration, a multi-container Docker Compose structure coordinates container dependencies, network configurations, volume mounts, and health-check boundaries. The orchestration file sets up isolated execution networks, mapping frontend and proxy containers to an outer public routing loop while restricting database services to an isolated internal persistence channel. Containers are assigned deterministic health checks (e.g., executing pg_isready inside the PostgreSQL image) to prevent application processes from accepting traffic before downstream data engines are fully active.

9.3 PostgreSQL 18 Production Provisioning

The relational database layer uses a dedicated container volume to ensure data persistence across container restarts. The container configuration optimizes PostgreSQL 18 performance based on available host hardware resources:

- **shared_buffers:** Set to 25% of total system memory to speed up data caching.
- **work_mem:** Configured to allocate sufficient RAM for complex analytical sorting operations directly in memory.
- **max_connections:** Capped to optimize resource allocation under heavy concurrency.

To safeguard historical merchant data, automated background routines trigger hourly logical backups, export compressed database snapshots to isolated cloud object storage, and clean up historical files based on retention policies.

9.4 Redis Memory Layer Management

The Redis instance operates as a high-performance in-memory cache and session state engine, running in a protected private network segment. The configuration enforces a strict maxmemory cap alongside a volatile-lru (Least Recently Used) eviction policy. This setup ensures that if memory limits are reached, the system automatically purges older cached API payloads while

protecting critical active session tokens and rate-limiting counters. Data persistence is managed using Append Only File (AOF) logging to allow rapid state recovery in the event of an unexpected host failure.

9.5 Nginx Reverse Proxy and Gateway Routing

Nginx functions as the edge reverse proxy and traffic manager for the deployment architecture. It handles inbound user traffic on port 443, applies SSL/TLS configurations, and routes requests across internal container streams using optimized configurations:

```
upstream backend_nodes {
  server bms-backend:3000 max_fails=3 fail_timeout=10s;
}

server {
  listen 443 ssl http2;
  server_name api.bms-enterprise.com;

  ssl_certificate     /etc/ssl/certs/bms_chain.crt;
  ssl_certificate_key /etc/ssl/private/bms_server.key;
  ssl_protocols       TLSv1.2 TLSv1.3;

  location / {
    proxy_pass http://backend_nodes;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }
}
```

This setup offloads cryptographic processing from application runtimes, manages connection pools efficiently, and includes compression features that minimize bandwidth consumption for static client assets.

9.6 Environment Variables Configurations

Application behavior is managed dynamically using externalized environment properties, preventing sensitive configuration parameters from being hardcoded inside compiled code paths. These configurations are managed through encrypted deployment variables and are split into three categories:

- **Infrastructure Identifiers:** Database connection strings (DATABASE_URL), Redis cluster paths, and active port assignments.
- **Cryptographic Keys:** JWT asymmetric secrets (JWT_SECRET), webhook signature keys, and API credentials for external payment gateways.
- **Global Overrides:** System environment modes (NODE_ENV=production), allowed CORS origin arrays, and notification configuration variables.

9.7 Automated Production Deployment Pipelines

The deployment workflow is fully automated through a continuous integration and continuous deployment (CI/CD) pipeline. When code changes are merged into the primary branch, automation runners execute the following steps:

- Run linting checks, type verification validations, and unit test suites.
- Compile code modules into optimized production Docker images.
- Push version-tagged images to a private secure container registry.
- Execute zero-downtime rolling updates to update active application containers seamlessly.
- Trigger automated database migrations through the Prisma schema engine.

9.8 System Scalability and High-Availability Models

To maintain low-latency response times as user volumes grow, the infrastructure supports both horizontal and vertical scaling paths. The NestJS and Next.js containers are designed to be completely stateless, allowing them to scale out across multiple nodes using an infrastructure load balancer to distribute user traffic. Database performance is optimized by separating transactional workloads from analytical reporting. Read replicas are deployed to handle reporting queries, offloading intensive aggregation tasks from the primary database cluster and ensuring consistent performance for core point-of-sale operations.

10. DESIGN DECISIONS

This chapter provides a detailed justification for the core architectural components selected for the Business Management System (BMS). Each technology choice is evaluated based on enterprise design parameters, development speed, and operational cost efficiency compared to available alternatives.

10.1 Presentation Tier Selection Rationale

10.1.1 Next.js Framework

The presentation architecture uses Next.js instead of a client-side Single Page Application (SPA) framework like standard React or Vue.js. Next.js provides hybrid Server-Side Rendering (SSR) and React Server Components (RSC) natively within its App Router model.

Architectural Trade-off Analysis: Standard SPAs download heavy JavaScript bundles before executing the initial Document Object Model (DOM) render, which delays interactive rendering on lower-specification hardware. Next.js processes initial layout states inside the backend deployment infrastructure, serving optimized HTML fragments directly to the browser client. This setup minimizes Time to Interactive (TTI) and First Contentful Paint (FCP) metrics, which is essential for ensuring a fast, uninterrupted user interface in high-throughput Point of Sale environments.

10.1.2 Tailwind CSS

The user interface styles are generated using Tailwind CSS instead of traditional runtime CSS-in-JS libraries (such as Styled-Components or Emotion) or legacy CSS preprocessors (SASS/LESS).

Architectural Trade-off Analysis: Runtime CSS-in-JS solutions introduce computational overhead by processing style modifications inside the browser script thread, which can cause frame drops during complex UI state transitions. Tailwind CSS evaluates the source codebase during compilation to generate a highly optimized, static utility style sheet. This compilation approach eliminates unused styles, drastically reduces network asset sizes, and ensures consistent rendering performance across all point-of-sale terminal configurations.

10.1.3 Shadcn UI

The application interface uses Shadcn UI rather than complete, pre-styled component frameworks like Material UI (MUI) or Ant Design.

Architectural Trade-off Analysis: Complete UI frameworks bundle restrictive style systems and large asset weights that are difficult to customize for highly optimized screen layouts. Shadcn UI provides unstyled, accessible primitives built on top of Radix UI. This allows the engineering team to copy accessible component code directly into the local repository, giving them complete structural control over DOM mechanics, state variables, and design overrides required to build clean, fast cashier workspaces.

10.2 Service Tier Selection Rationale

10.2.1 NestJS Framework

The backend application engine is built on the NestJS framework within the Node.js runtime environment, rather than choosing alternative architectures like Express.js or Python-based Django frameworks.

Architectural Trade-off Analysis: Express.js provides minimal structural guidelines, which frequently leads to disorganized and difficult-to-maintain codebases in complex enterprise projects. NestJS enforces a consistent, modular architecture out of the box. Its advanced dependency injection engine decouples controllers, domain logic providers, and persistence layers cleanly, simplifying module isolation and enabling deterministic unit testing across the enterprise lifecycle.

10.2.2 TypeScript

The entire software stack uses TypeScript for development rather than native JavaScript.

Architectural Trade-off Analysis: JavaScript's loose dynamic typing can hide structural data mismatches, leading to unexpected runtime exceptions that disrupt live multi-tenant business systems. TypeScript introduces strict compile-time verification across the application lifecycle. By defining immutable data contracts and interfaces that span from frontend forms to backend API boundaries and ORM data structures, the team catches structural type mismatches at compilation, ensuring high system stability and reducing debugging overhead.

10.3 Persistence and Caching Tier Selection Rationale

10.3.1 PostgreSQL 18

The primary persistence engine relies on PostgreSQL 18, rather than opting for alternative document-oriented NoSQL models like MongoDB.

Architectural Trade-off Analysis: NoSQL databases prioritize loose horizontal schema layouts over absolute mathematical consistency, which makes them unsuitable for handling interdependent transactional domains like multi-tenant financial ledgers, warehouse inventory tracing, and tax calculations. PostgreSQL 18 offers reliable ACID compliance, advanced transactional isolation levels, and row-level locking mechanisms. These features ensure data consistency and prevent race conditions or ledger mismatch anomalies under heavy multi-user write stress.

10.3.2 Prisma ORM

Database access and query mapping are managed through Prisma ORM instead of TypeORM or raw SQL query drivers.

Architectural Trade-off Analysis: Traditional ORMs map database tables to mutable runtime code models, which can hide structural changes and cause unexpected object synchronization failures during compilation. Prisma ORM parses a single, declarative schema file to automatically generate type-safe TypeScript query interfaces. This setup ensures that if a database column is changed, any mismatched code blocks across services are flagged during compilation, reducing runtime errors while still allowing developers to write optimized raw SQL queries for complex reporting tasks.

10.3.3 Redis Cache Cluster

Redis is deployed as an in-memory key-value data structure store, functioning as an auxiliary acceleration layer alongside the relational database core.

Architectural Trade-off Analysis: Querying a relational database repeatedly to verify active authentication tokens, manage API rate-limiting tables, or fetch static catalog items can overload the persistent storage engine, reducing transactional throughput. Redis handles these operations in memory, achieving microsecond read response times. This offloads repetitive query strain from the primary PostgreSQL instance, keeping the main database engine focused on processing critical transactional writes.

10.4 Infrastructure Tier Selection Rationale

10.4.1 Docker

Every operational component is isolated inside a standard Docker container rather than being provisioned directly on virtual machine instances.

Architectural Trade-off Analysis: Deploying applications directly on host machines can cause runtime failures due to minor differences in underlying library versions, configuration setups, or operating system environments between development and production. Docker packages the application code alongside its precise runtime dependencies into an isolated software container. This containerization guarantees identical execution behavior across every host machine, isolates software packages cleanly, and simplifies server resource allocation.

10.4.2 Docker Compose

The local development environment and baseline deployment structures are orchestrated through Docker Compose rather than deploying a heavy cloud Kubernetes stack immediately at launch.

Architectural Trade-off Analysis: Kubernetes provides powerful multi-region cluster automation but introduces immense deployment complexity, configuration overhead, and infrastructure costs that can slow down initial development for small and medium enterprises. Docker Compose offers a clean, lightweight orchestration model using structured YAML files. It allows engineers to manage multi-container application stacks, configure internal private networks, and define persistent storage volumes easily without introducing unneeded operational complexity during early deployment phases.

10.4.3 Nginx Web Server

Nginx is deployed at the perimeter edge of the infrastructure cluster, functioning as a reverse proxy and gatekeeper.

Architectural Trade-off Analysis: Routing public user traffic directly to Node.js application containers exposes application processes to direct connection bottlenecks, resource-intensive TLS handshake computations, and direct security attacks. Nginx is optimized to handle thousands of concurrent connections efficiently. It manages TLS/SSL certificate termination at the perimeter, serves static web assets directly to users, and filters incoming traffic to protect backend service layers from volumetric security threats.

11. FUTURE ENHANCEMENTS

This chapter outlines the long-term technological roadmap for the Business Management System (BMS). These forward-looking modules are designed to transition the application from a core Point of Sale and inventory platform into an integrated, intelligent enterprise utility.

11.1 Native Mobile Application

To untether business operations from fixed desktop checkout counters, a native mobile application expansion is planned using a cross-platform framework (such as React Native or Flutter). This application will communicate with the same NestJS backend REST API endpoints, allowing warehouse staff to perform real-time inventory cycle counts directly on the floor and enabling store managers to monitor live sales performance dashboards remotely.

11.2 Hardware-Integrated Barcode Scanning Pipelines

The system will expand its peripheral hardware capabilities by introducing native support for continuous hardware-integrated barcode scanning. The front-end user interface will bind directly to dedicated USB and Bluetooth HID (Human Interface Device) scanning hardware. By capturing rapid keyboard-emulated input streams and passing them to an optimized client-side buffer lookup service, cashiers will be able to append products to an active cart instantly without touching a mouse or manual keyboard interface.

11.3 Multi-Branch Enterprise Synchronization

The platform's logical architecture will expand to support multi-branch operations under a single parent business tenant. The database schema will incorporate a branches entity table, establishing a hierarchical structure where a single business entity can operate independent retail outlets and storage warehouses. Inventory records will track stock levels on a per-branch basis, allowing managers to coordinate inter-branch stock transfers, compare performance metrics across locations, and manage centralized supply procurement.

11.4 Asynchronous Offline Mode for Resilient POS Operations

To protect retailers against business disruptions caused by internet connectivity drops, the Point of Sale interface will implement an asynchronous offline operational mode. Leveraging modern browser capabilities — including Service Workers for asset caching and an IndexedDB database for client-side storage — the POS workspace will remain fully functional without a live network connection. Cashiers can continue scanning products and finalizing cash checkouts; transaction records will save locally to IndexedDB and automatically sync with the cloud NestJS backend once connectivity is restored.

11.5 Omni-Channel Automated SMS Notifications

The system will integrate with cloud communications platforms (such as Twilio or Africa's Talking) to run automated SMS notification workflows. This messaging pipeline will distribute

automated receipt summaries to customers upon checkout, dispatch low-stock alerts to managers when inventory drops below restock thresholds, and send automated payment reminders to credit clients. The notification service will execute inside isolated background worker threads to keep front-end point-of-sale workflows fast and responsive.

11.6 Double-Entry Accounting Architecture Integration

To provide businesses with a complete financial overview, the core sales and procurement modules will integrate with an automated double-entry accounting engine. Every transaction — whether a finalized consumer sale, a supplier payment, or a stock adjustment — will automatically generate matching debit and credit ledger entries across specialized accounts (e.g., Accounts Receivable, Inventory Valuation, Cost of Goods Sold, and Cash Assets). This automation eliminates manual accounting entry errors and allows businesses to generate instant, audit-ready balance sheets and profit-and-loss statements.

11.7 Predictive Machine Learning for Sales Forecasting

The system will introduce an intelligent analytics layer utilizing machine learning models to forecast future sales trends. By analyzing historical transaction records, seasonal buying patterns, and regional economic indicators, a predictive model (such as an LSTM network or Prophet forecasting engine) will project future revenue trends. These analytical predictions will render directly within the manager’s dashboard, allowing businesses to optimize staffing levels and adjust marketing strategies proactively.

11.8 Automated ML-Driven Inventory Replenishment Optimization

The predictive analytics engine will extend directly into the supply chain to automate inventory restock planning. An intelligent inventory forecasting module will compute dynamic, automated safety stock thresholds and optimal reorder points for every catalog item based on historical sales velocity and supplier fulfillment lead times. When the system detects that an item’s stock is projected to fall below safety thresholds before the next delivery cycle, it will automatically compile a draft purchase order for the exact replacement volume, reducing capital tied up in excess warehouse stock while preventing costly out-of-stock scenarios.

APPENDICES

Appendix A – Project Folder Structure

```
bms-root/
├── apps/
│   ├── bms-backend/
│   │   ├── src/
│   │   │   ├── auth/           # Authentication & JWT validation module
│   │   │   ├── business/       # Multi-tenant corporate configuration
│   │   │   ├── inventory/      # Stock management & allocation logic
│   │   │   ├── products/       # Catalog, SKUs, and barcode tracking
│   │   │   ├── sales/          # POS checkout processing engine
│   │   │   ├── users/          # User accounts & RBAC definition
│   │   │   ├── prisma/         # Database instantiation & seed files
│   │   │   └── main.ts         # Backend application entry point
│   │   └── Dockerfile
│   └── bms-frontend/
│       ├── src/
│       │   ├── app/            # Next.js App Router (Layouts & Pages)
│       │   ├── components/     # Reusable Shadcn UI visual atoms
│       │   ├── hooks/          # Custom state synchronization utilities
│       │   └── services/       # REST API client wrapper modules
│       └── Dockerfile
├── libs/
│   └── bms-shared-contracts/  # Shared TypeScript validation DTOs
├── docker-compose.yml          # Container orchestration manifest
└── README.md
```

Appendix B – Core Database Migration Scripts

```
-- Core structural initialization script for PostgreSQL 18
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. Create Businesses Table
CREATE TABLE businesses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  tax_number VARCHAR(50) UNIQUE,
  currency VARCHAR(3) NOT NULL DEFAULT 'USD',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 2. Create Users Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_id UUID NOT NULL REFERENCES businesses(id) ON DELETE CASCADE,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL CHECK (role IN ('ADMIN', 'MANAGER', 'CASHIER')),
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Create optimized B-Tree index for authentication lookups
CREATE INDEX idx_users_email_search ON users(email);
```

Appendix C – Engineering Naming Conventions

To maintain absolute consistency across all development teams, the complete codebase must adhere strictly to the following architectural naming conventions:

- **Database Tables and Columns:** Must use lowercase snake_case syntax exclusively (e.g., sale_items, low_stock_threshold). Foreign key column links must combine the target table's singular name with an _id suffix (e.g., business_id).
- **TypeScript Files and Classes:** File structural components must use lowercase kebab-case syntax denoting their structural role (e.g., product-catalog.controller.ts, create-user.dto.ts). Internal class declarations must use strict PascalCase matching their file prefix (e.g., ProductCatalogController).
- **Variables and Service Methods:** Internal procedural names, state objects, and method functions must use standard camelCase syntax (e.g., calculateTotalTaxAmount(), activeCartItems).

Appendix D – Glossary

- **Tenant:** An independent commercial enterprise account within the system, whose data structures are kept completely isolated from all other organizations.
- **Stock Keeping Unit (SKU):** A unique alphanumeric identifier assigned to a specific product variant to track inventory movements accurately.
- **Race Condition:** A system anomaly where concurrent data access operations attempt to modify the same database records simultaneously, potentially leading to inconsistent or corrupted state information.
- **Idempotency:** An API processing principle ensuring that an operation produces the exact same system state outcome even if it is executed multiple times with identical parameters.

Appendix E – Acronyms

- **SDD:** Software Design Description
- **ERD:** Entity Relationship Diagram
- **REST:** Representational State Transfer
- **DTO:** Data Transfer Object
- **API:** Application Programming Interface
- **JSON:** JavaScript Object Notation
- **TTL:** Time To Live
- **HID:** Human Interface Device
- **UUID:** Universally Unique Identifier